

TRƯỜNG ĐẠI HỌC THỦY LỢI  
KHOA CÔNG NGHỆ THÔNG TIN  
BỘ MÔN ĐỒ ÁN TRÍ TUỆ NHÂN TẠO VÀ KHOA HỌC DỮ LIỆU

-----oO-----



BÀI TẬP LỚN

MÔN HỌC: XỬ LÝ NGÔN NGỮ TỰ NHIÊN

Tên đề tài: Sửa lỗi ngữ pháp tự động bằng các mô hình học máy  
truyền thống và hiện đại

Giáo viên hướng dẫn: Nguyễn Quang Hoan

Lớp: 64TTNT2

Tên các thành viên: Nguyễn Đình Huân

Hà Nội

## Mục lục

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| BÀI TẬP LỚN .....                                                                           | 1  |
| Lời cảm ơn .....                                                                            | 3  |
| Giới thiệu .....                                                                            | 4  |
| A. Tập dữ liệu .....                                                                        | 4  |
| 1. Tập dữ liệu JFLEG - Johns Hopkins Fluency-Extended GUG<br>Corpus. ....                   | 5  |
| 2. Mục đích và Ứng dụng. ....                                                               | 5  |
| 3. Nội dung .....                                                                           | 5  |
| 4. Đặc điểm chính: .....                                                                    | 5  |
| B. Các thuật toán sử dụng .....                                                             | 7  |
| 1. Mô hình truyền thống Rule-based Random Forest GEC Model .....                            | 7  |
| 2. Mô hình T5 Transformer cho nhiệm vụ sửa lỗi ngữ pháp (Grammar<br>Error Correction) ..... | 12 |
| C. Các bước thực hiện .....                                                                 | 19 |
| 1. Rule-based Random Forest GEC Model .....                                                 | 19 |
| 2. Mô hình T5 Transformer .....                                                             | 31 |
| D. Kết quả và phân tích .....                                                               | 36 |
| E. Kết luận và định hướng tương lai .....                                                   | 38 |
| Tổng kết .....                                                                              | 40 |
| Lời kết .....                                                                               | 41 |

## Lời cảm ơn

Trước hết, em xin trân trọng gửi lời biết ơn sâu sắc nhất đến toàn thể quý Thầy cô trong Bộ môn Trí tuệ nhân tạo – Khoa Công nghệ Thông tin, những người đã tận tâm giảng dạy, định hướng và truyền thụ cho em nền tảng vững chắc về xử lý ngôn ngữ tự nhiên, kiến trúc Transformer và những kỹ thuật học sâu tiên tiến. Nhờ sự chỉ dẫn tỉ mỉ của thầy cô, em đã:

- Được lĩnh hội toàn diện cơ chế attention, encoder–decoder và quy trình fine-tune các mô hình ngôn ngữ lớn, giúp em hiểu rõ từng bước trong chu trình huấn luyện.
- Thành thạo từ khâu tiền xử lý dữ liệu: tiền xử lý văn bản, tokenization, tạo tập dữ liệu huấn luyện/đánh giá, đến việc xây dựng pipeline cho việc training và inference.
- Làm chủ các chỉ số đánh giá chất lượng mô hình như BLEU, F1, Exact Match, đồng thời biết cách diễn giải kết quả để cải tiến mô hình.
- So sánh, phân tích ưu – nhược điểm giữa nhiều phương pháp (từ RandomForest cơ bản đến T5-small fine-tune), từ đó rút ra những bài học quý giá cho việc thiết kế giải pháp tối ưu.
- củng cố khả năng lập trình và quản lý dự án: tổ chức code module hóa, debug hiệu quả, áp dụng kỹ thuật tối ưu hyper-parameters, early-stopping và gradient clipping để đảm bảo mô hình hội tụ ổn định.
- Rèn luyện tinh thần tự học, tinh thần nghiên cứu khoa học qua việc chủ động tìm kiếm tài liệu, thử nghiệm thuật toán, phân tích kết quả thực nghiệm và liên tục hoàn thiện đề tài.

Em xin gửi lời cảm ơn chân thành đến Ban chủ nhiệm Khoa Công nghệ Thông tin và Ban Giám hiệu Nhà trường đã tạo điều kiện thuận lợi về cơ sở vật chất, trang thiết bị máy tính và môi trường học tập năng động, sáng tạo. Đồng thời, em cũng trân trọng cảm ơn các cán bộ phòng thí nghiệm, các anh/chị kỹ thuật viên và các bạn sinh viên trong nhóm nghiên cứu đã luôn sẵn sàng hỗ trợ, trao đổi ý tưởng và chia sẻ kinh nghiệm thực tế.

Cuối cùng, em xin cảm ơn gia đình và bạn bè – những người đã đồng hành, tiếp thêm động lực và dành cho em sự ủng hộ tinh thần to lớn trong suốt quá trình thực hiện đề tài. Sự khích lệ và tin tưởng ấy chính là động lực giúp em vượt qua khó khăn, hoàn thành công trình nghiên cứu này. Em mong rằng kết quả đạt được sẽ góp phần nhỏ bé nhưng thiết thực vào sự phát triển chung của lĩnh vực Xử lý ngôn ngữ tự nhiên tại Khoa, cũng như mở ra những hướng ứng dụng mới trong tương lai.

Một lần nữa, em xin chân thành cảm ơn và kính chúc quý Thầy cô, các cán bộ và bạn bè luôn mạnh khỏe, hạnh phúc và gặt hái nhiều thành công trong sự nghiệp và cuộc sống.

## **Giới thiệu**

Trong thời đại số hóa ngày nay, giao tiếp bằng văn bản đóng vai trò ngày càng quan trọng. Tuy nhiên, lỗi ngữ pháp là một vấn đề phổ biến có thể ảnh hưởng đến sự rõ ràng và chuyên nghiệp của văn bản. Đề tài này tập trung vào việc xây dựng và đánh giá các mô hình sửa lỗi ngữ pháp tự động sử dụng các kỹ thuật học máy và học sâu. Chúng tôi đã triển khai mô hình T5-small dựa trên transformer, llms, và các mô hình truyền thống. Mục tiêu là xác định phương pháp hiệu quả nhất để tự động phát hiện và sửa chữa các lỗi ngữ pháp trong văn bản tiếng Anh. Các mô hình dựa trên mạng nơ-ron, đặc biệt là các mô hình transformer, đã đạt được kết quả đột phá trong lĩnh vực này. Tuy nhiên, việc lựa chọn mô hình phù hợp và tối ưu hóa hiệu suất vẫn là một thách thức. Với sự gia tăng của khối lượng văn bản được tạo ra hàng ngày trên internet và các nền tảng kỹ thuật số, nhu cầu về các công cụ hỗ trợ viết hiệu quả ngày càng trở nên cấp thiết. Hệ thống sửa lỗi ngữ pháp tự động không chỉ giúp cải thiện chất lượng văn bản mà còn tiết kiệm thời gian và công sức cho người viết. Đề tài này xuất phát từ nhu cầu thực tế đó. Chúng tôi đặt mục tiêu nghiên cứu và ứng dụng các kỹ thuật học máy và học sâu hiện đại để giải quyết bài toán sửa lỗi ngữ pháp trong văn bản tiếng Anh. Chúng tôi sẽ tiến hành xây dựng các mô hình, huấn luyện chúng trên dữ liệu phù hợp và đánh giá hiệu quả của từng mô hình để rút ra những kết luận hữu ích. Thông qua việc so sánh các phương pháp khác nhau, chúng tôi hy vọng sẽ đóng góp vào việc phát triển các hệ thống sửa lỗi ngữ pháp tiếng Anh chính xác và hiệu quả hơn.

### **A. Tập dữ liệu**

# 1. Tập dữ liệu JFLEG - Johns Hopkins Fluency-Extended GUG Corpus.

**Nguồn gốc:** Được công bố bởi Đại học Johns Hopkins, JFLEG là một tập dữ liệu dùng trong các bài toán sửa lỗi cú pháp tiếng anh (Grammatical Error Correction - GEC).

**2. Mục đích và Ứng dụng:** Tập dữ liệu này đóng vai trò là một tiêu chuẩn vàng (gold standard benchmark) để phát triển và đánh giá các hệ thống GEC. Mục tiêu chính của nó là đánh giá độ lưu loát (mức độ tự nhiên như người bản xứ) và tính ngữ pháp của các văn bản đã được sửa chữa.

**3. Nội dung:** Tập dữ liệu chứa các câu gốc được viết bởi những người học tiếng Anh. Đối với mỗi câu nguồn, có bốn phiên bản sửa lỗi được viết bởi con người.

Cấu trúc dữ liệu Mỗi trường hợp trong tập dữ liệu được cấu trúc như sau :

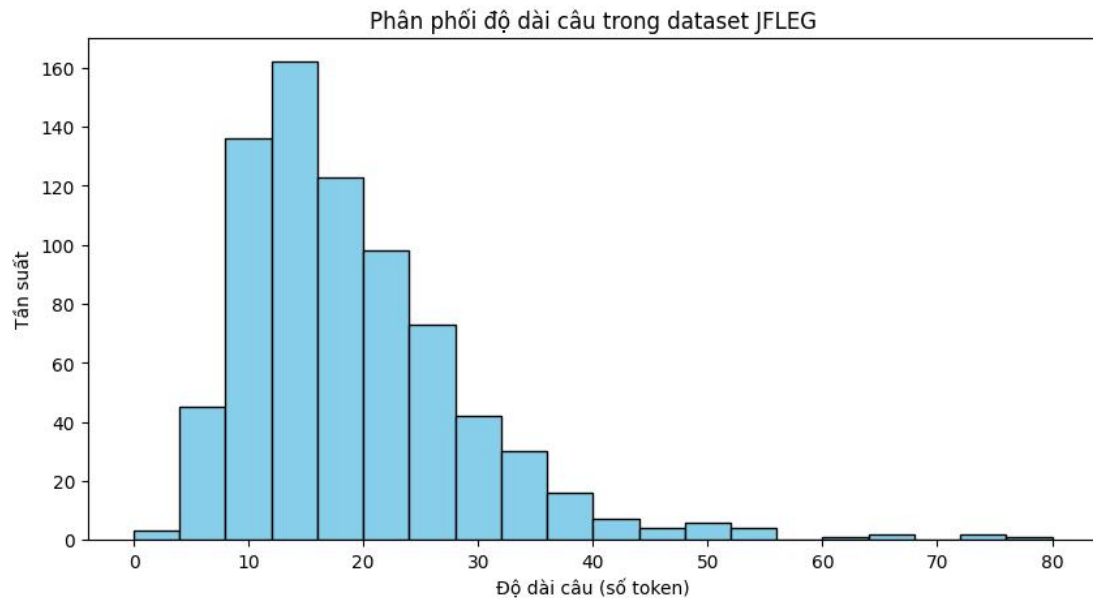
sentence: Trường này chứa câu gốc do người học tiếng Anh viết.

corrections: Trường này chứa một chuỗi gồm bốn phiên bản đã được sửa chữa của sentence, được cung cấp bởi những người chủ thích là con người. Thứ tự của các chú thích này là nhất quán.

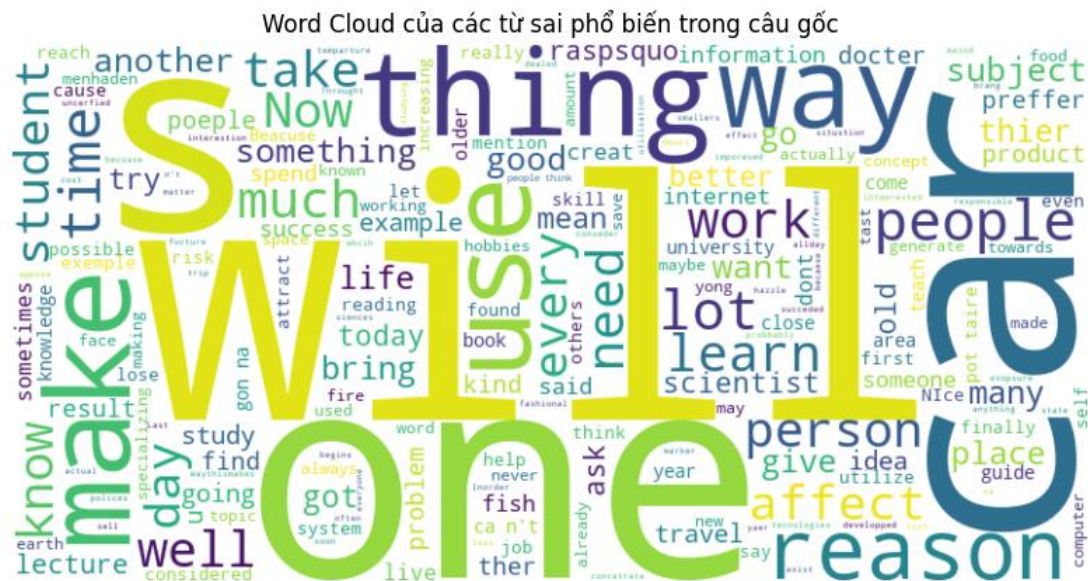
## 4. Đặc điểm chính:

- Kích thước: Tập dữ liệu bao gồm tổng cộng 1511 câu trong tập train, 754 câu trong tập test.
- Nhiều bản sửa lỗi: Một đặc điểm quan trọng là mỗi câu nguồn có bốn phiên bản sửa lỗi tương ứng, cho phép đánh giá mạnh mẽ hơn các hệ thống GEC.
- Ngôn ngữ: Tập dữ liệu bằng tiếng Anh và bao gồm các văn bản từ cả người viết bản xứ và người học ngôn ngữ thứ hai (L2).
- Định dạng: Dữ liệu có sẵn ở định dạng parquet.

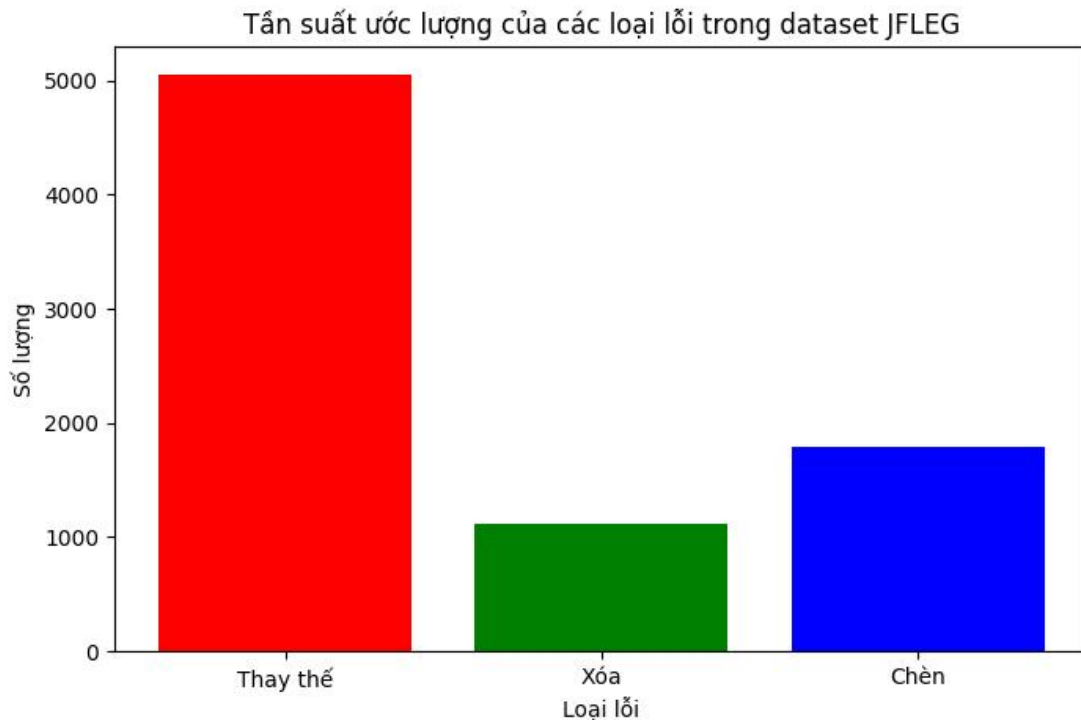
| sentence<br>string , lengths                                                                                                            | corrections<br>sequence                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| So I think we can not live if old people could not find sciences and technologies and they did not developped .                         | [ "So I think we would not be alive if our ancestors did not develop sciences and technologies . " , "So I think we could not...       |
| For not use car .                                                                                                                       | [ "Not for use with a car . " , "Do not use in the car . " , "Car not for use . " , "Can not use the car . " ]                         |
| Here was no promise of morning except that we looked up through the trees we saw how low the forest had swung .                         | [ "Here was no promise of morning , except that we looked up through the trees , and we saw how low the forest had swung . " , ...     |
| Thus even today sex is considered as the least important topic in many parts of India .                                                 | [ "Thus , even today , sex is considered as the least important topic in may parts of India . " , "Thus , even today , sex is...       |
| image you salf you are wark in factory just to do one thing like pot taire on car if they fire you you will destroy , becouse u don...  | [ "Imagine yourself you are working in factory just to do one thing like put air a on car if they fire you you will be destroyed , ... |
| They draw the consumers , like me , to purchase this great product with all these amazing ingredients and all that but actually they... | [ "They draw in the consumers , like me , to purchase a great product with amazing ingredients , but actually , they just make u...    |



Biểu đồ cho thấy phần lớn câu trong dataset JFLEG có độ dài từ 10 đến 20 token, với tần suất cao nhất rơi vào khoảng này. Phân phối có dạng lệch phải, nghĩa là càng ít câu có độ dài lớn. Điều này cho thấy JFLEG chủ yếu chứa các câu ngắn đến trung bình, phù hợp cho các mô hình xử lý ngôn ngữ tự nhiên với giới hạn độ dài đầu vào khoảng 30–50 token, giúp tối ưu hiệu suất tính toán và bộ nhớ khi huấn luyện.



Các từ xuất hiện với kích thước lớn như “will”, “one”, “make”, “thing”, “way” cho thấy đây là những từ thường xuyên mắc lỗi – có thể do sử dụng sai ngữ pháp, mạo từ, thì hoặc cách kết hợp từ. Từ đó, ta có thể rút ra rằng mô hình sửa lỗi cần chú ý nhiều đến động từ khuyết thiếu, danh từ trừu tượng, và cấu trúc câu với chủ thể không rõ ràng, vốn là lỗi ngôn ngữ học điển hình của người học tiếng Anh.



Biểu đồ cho thấy trong dataset JFLEG, lỗi thay thế từ chiếm tỷ lệ cao nhất với hơn 5000 trường hợp, vượt xa lỗi chèn và xóa. Điều này cho thấy phần lớn lỗi trong văn bản gốc là do người viết dùng sai từ hoặc cấu trúc cần được thay thế, phản ánh đặc điểm lỗi thường gặp của người học tiếng Anh. Vì vậy, mô hình sửa lỗi cần được huấn luyện đặc biệt tốt cho loại lỗi thay thế để đạt hiệu quả cao.

## B. Các thuật toán sử dụng

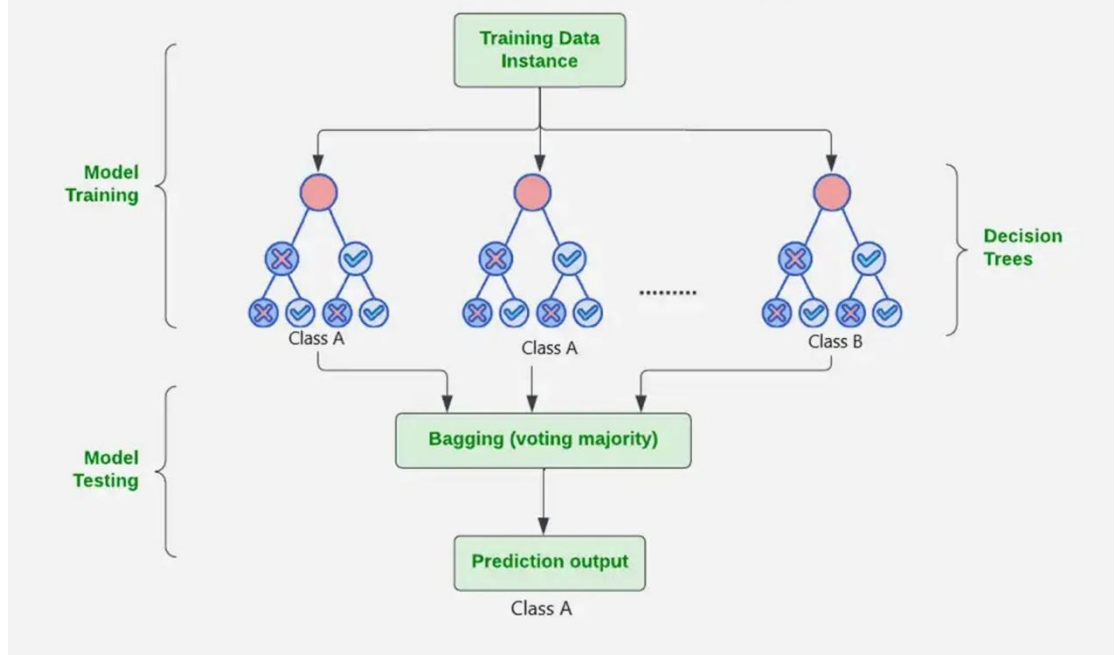
### 1. Mô hình truyền thống Rule-based Random Forest GEC Model

- Thuật toán Machine Learning chính:** RandomForestClassifier  
(Mô hình sửa lỗi ngữ pháp dựa trên luật kết hợp RandomForest)

#### - Giới thiệu chung

Random Forest là một thuật toán học máy thuộc nhóm ensemble learning, cụ thể là bagging, được xây dựng từ nhiều cây quyết định (Decision Trees). Thay vì chỉ dựa vào một mô hình duy nhất, Random Forest kết hợp nhiều mô hình yếu để tạo thành một mô hình mạnh hơn, giúp cải thiện độ chính xác và giảm hiện tượng quá khớp (overfitting)

# Random Forest Algorithm in Machine Learning



## - Đầu vào và đầu ra

Đầu vào: Một bảng dữ liệu gồm các đặc trưng (features), có thể là số hoặc phân loại.

Đầu ra:

Với bài toán phân loại: nhãn (label) thuộc một trong nhiều lớp.

Với bài toán hồi quy: một giá trị số liên tục.

Đầu ra được tính bằng cách:

Phân loại: bỏ phiếu đa số từ các cây.

Hồi quy: tính trung bình kết quả dự đoán từ các cây.

## - Cơ chế hoạt động

Bagging (Bootstrap Aggregating): Mỗi cây được huấn luyện trên một tập con ngẫu nhiên của dữ liệu gốc (có lặp).

Random feature selection: Tại mỗi nút chia trong cây, chỉ một tập con ngẫu nhiên các đặc trưng được chọn để tăng tính ngẫu nhiên và giảm tương quan giữa các cây.

Tổng hợp kết quả:

Với phân loại: kết quả là lớp có nhiều cây chọn nhất (majority vote).

Với hồi quy: kết quả là trung bình các dự đoán của tất cả các cây.

## - Hàm mất mát (Loss function)

Không được tối ưu trực tiếp như mạng nơ-ron, vì mỗi cây học riêng biệt.

Trong từng cây:



Phân loại: sử dụng Gini Impurity hoặc Entropy để chia nhánh.

Hồi quy: sử dụng Mean Squared Error (MSE) tại mỗi nút chia.

- Ưu điểm

Khả năng tổng quát hóa tốt, ít overfitting.

Xử lý tốt dữ liệu không tuyến tính, dữ liệu có nhiễu.

Hỗ trợ cả phân loại và hồi quy.

Không cần chuẩn hóa dữ liệu.

Có thể đánh giá tầm quan trọng của từng đặc trưng.

Hoạt động tốt với dữ liệu có thiếu giá trị (missing values).

- Nhược điểm

Kích thước mô hình lớn, tiêu tốn bộ nhớ.

Tốc độ suy luận chậm nếu có quá nhiều cây.

Ít trực quan, khó diễn giải hơn so với cây quyết định đơn.

Không phù hợp với dữ liệu có số chiều quá lớn so với số lượng mẫu.

- Các tham số quan trọng

n\_estimators: Số lượng cây trong rừng.

max\_depth: Độ sâu tối đa của mỗi cây.

max\_features: Số lượng đặc trưng được chọn ngẫu nhiên tại mỗi split.

min\_samples\_split: Số mẫu tối thiểu để chia một node.

min\_samples\_leaf: Số mẫu tối thiểu tại một lá.

bootstrap: Có sử dụng lấy mẫu lại hay không (True/False).

b) **Thuật toán xử lý text và tính độ tương đồng:**

Sequence Matcher ( tính similarity giữa các chuỗi )

Levenshtein Distance (tính khoảng cách giữa 2 chuỗi)

BLEU score( đánh giá chất lượng sửa lỗi)

+ Sentence BLEU

+ Corpus BLEU

+ Smoothing method từ SmoothingFunction

c) **Phương pháp phát hiện và sửa lỗi:**

***Subject-verb agreement detection***

- Khái niệm: Subject-verb agreement detection (phát hiện sự hòa hợp giữa chủ ngữ và động từ) là một bài toán trong xử lý ngôn ngữ tự nhiên (NLP) nhằm xác định xem một câu có lỗi ngữ pháp liên quan đến việc chủ ngữ và động từ có đúng dạng số (số ít/số nhiều) và ngôi (person) hay không.

Ví dụ:

Đúng: The dog barks. (Chủ ngữ số ít, động từ chia số ít)

Sai: The dog bark. (Chủ ngữ số ít, động từ chia sai ở số nhiều)

***Article error detection:***

Khái niệm: Article error detection là bài toán phát hiện lỗi sử dụng mạo từ trong câu, thường tập trung vào ba mạo từ chính trong tiếng Anh:

- “a”
- “an”
- “the”
- trường hợp không dùng mạo từ (zero article).

Ví dụ về lỗi:

- ✗ She adopted a animal. → ✓ She adopted an animal.
- ✗ He went to the home late. → ✓ He went home late.
- ✗ She is a best player. → ✓ She is the best player.

### **Preposition error detection**

Khái niệm: Preposition error detection là nhiệm vụ phát hiện và sửa lỗi giới từ (prepositions) trong câu. Giới từ là từ dùng để chỉ mối quan hệ về thời gian, vị trí, phương hướng, nguyên nhân... giữa các thành phần trong câu (ví dụ: in, on, at, to, for, with, by, of).

Ví dụ lỗi phổ biến:

- ✗ I am interested on music. → ✓ I am interested in music.
- ✗ He arrived to the airport. → ✓ He arrived at the airport.

### **Spelling error detection**

Khái niệm: Spelling error detection là nhiệm vụ phát hiện các lỗi chính tả trong văn bản – tức là khi một từ bị viết sai so với từ đúng trong từ điển của ngôn ngữ. Nhiệm vụ này là một phần cơ bản nhưng cực kỳ quan trọng trong xử lý ngôn ngữ tự nhiên (NLP), đặc biệt trong các hệ thống kiểm tra ngữ pháp và chỉnh sửa văn bản.

Ví dụ lỗi chính tả:

- ✗ enviroment → ✓ environment
- ✗ teh → ✓ the
- ✗ reserch → ✓ research

#### **d) Phương pháp lựa chọn correction:**

Random selection: Phương pháp chọn lựa ngẫu nhiên

#### **e) Tiền xử lý và Feature Engineering:**

##### ***Word tokenization (NLTK)***

Word tokenization là quá trình tách một chuỗi văn bản thành các đơn vị nhỏ hơn gọi là tokens – thông thường là từ. Đây là bước tiền xử lý quan trọng trong xử lý ngôn ngữ tự nhiên (NLP), vì hầu hết các mô hình và thuật toán NLP đều hoạt động trên cấp độ từ.

NLTK (Natural Language Toolkit) là một thư viện Python phổ biến hỗ trợ mạnh mẽ cho việc tokenization, bao gồm cả word tokenization và sentence tokenization.

Mục tiêu: Tách một đoạn văn bản đầu vào thành danh sách các từ, loại bỏ khoảng trắng và xử lý đúng các ký tự đặc biệt, dấu câu, từ viết tắt...

Ví dụ:

Input: "Don't hesitate to ask questions!"

Output: ['Do', 'n't', 'hesitate', 'to', 'ask', 'questions', '!']

### ***POS tagging (NLTK)***

Khái niệm: POS tagging (Part-of-Speech tagging) là quá trình gán nhãn từ loại (danh từ, động từ, tính từ, trạng từ, v.v.) cho từng từ (token) trong một câu văn bản. Đây là một bước quan trọng trong tiền xử lý ngôn ngữ tự nhiên, hỗ trợ cho các nhiệm vụ như phân tích cú pháp, nhận dạng thực thể (NER), phân tích ngữ nghĩa, và dịch máy.

Xác định loại ngữ pháp (từ loại) của từng từ trong câu:

Ví dụ:

| Tag | Ý nghĩa                     | Ví dụ                     |
|-----|-----------------------------|---------------------------|
| NN  | Danh từ thường              | <i>dog, library</i>       |
| NNP | Danh từ riêng               | <i>John, NLTK</i>         |
| VB  | Động từ nguyên mẫu          | <i>go, run</i>            |
| VBD | Động từ quá khứ             | <i>went, ran</i>          |
| JJ  | Tính từ                     | <i>big, beautiful</i>     |
| RB  | Trạng từ                    | <i>quickly, silently</i>  |
| IN  | Giới từ / liên từ phụ thuộc | <i>in, on, because of</i> |
| DT  | Mạo từ                      | <i>a, the</i>             |
| PRP | Đại từ nhân xưng            | <i>he, she, they</i>      |

.....More

### ***N-gram extraction (bigrams, trigrams)***

N-gram là một chuỗi gồm n từ liên tiếp trong một văn bản hoặc câu. Việc trích xuất N-gram là một kỹ thuật tiền xử lý phổ biến trong NLP, giúp mô hình học được ngữ cảnh cục bộ, mối liên hệ giữa các từ và cải thiện hiệu quả của các tác vụ như phân loại văn bản, sinh văn bản, và dự đoán từ.

| Loại n-gram   | Ví dụ với câu: "I love natural language processing"              |
|---------------|------------------------------------------------------------------|
| Unigram (n=1) | ['I', 'love', 'natural', 'language', 'processing']               |
| Bigram (n=2)  | [('I', 'love'), ('love', 'natural'), ...]                        |
| Trigram (n=3) | [('I', 'love', 'natural'), ('love', 'natural', 'language'), ...] |

### ***Case preservation***

Case preservation (bảo toàn chữ hoa – chữ thường) là quá trình giữ nguyên định dạng viết hoa/viết thường của từ trong khi xử lý văn bản. Đây là một khía cạnh quan trọng trong NLP vì thông tin về chữ hoa thường mang ý nghĩa ngữ pháp, ngữ nghĩa hoặc phong cách trong câu.

| Chức năng của case    | Vai trò trong NLP                              |
|-----------------------|------------------------------------------------|
| Viết hoa tên riêng    | Giúp xác định thực thể (John, Google, Vietnam) |
| Viết hoa đầu câu      | Hỗ trợ phân tách câu đúng                      |
| Viết hoa để nhấn mạnh | Phân tích cảm xúc, ý đồ tác giả (STOP, NEVER)  |
| Phân biệt từ đồng âm  | Polish (người Ba Lan) vs polish (đánh bóng)    |

f) **Metrics đánh giá:**

Exact Match Accuracy: Chỉ số chính xác hoàn toàn

F1-score: Chỉ số nhận diện đúng vị trí, cách sửa sai cú pháp

Average Levenshtein Distance: Khoảng cách từ bị sửa với từ chuẩn (càng lớn -> sửa càng bị quá tay tức sai nhiều chỗ không cần sửa)

BLEU Score: Chỉ số đánh giá chất lượng bản dịch hoặc văn bản sinh ra trong NLP.

g) **Các kỹ thuật khác:**

StandardScaler: Chuẩn hóa dữ liệu

DictVectorizer: Chuyển đổi features dạng dict sang matrix

Train-test split: Chia tập dữ liệu (train, val, test)

Pipeline: xây dựng pipeline xử lý

h) **Phương pháp ensemble và parameter tuning:**

RandomForestClassifier với các tham số (từ GridSearchCV):

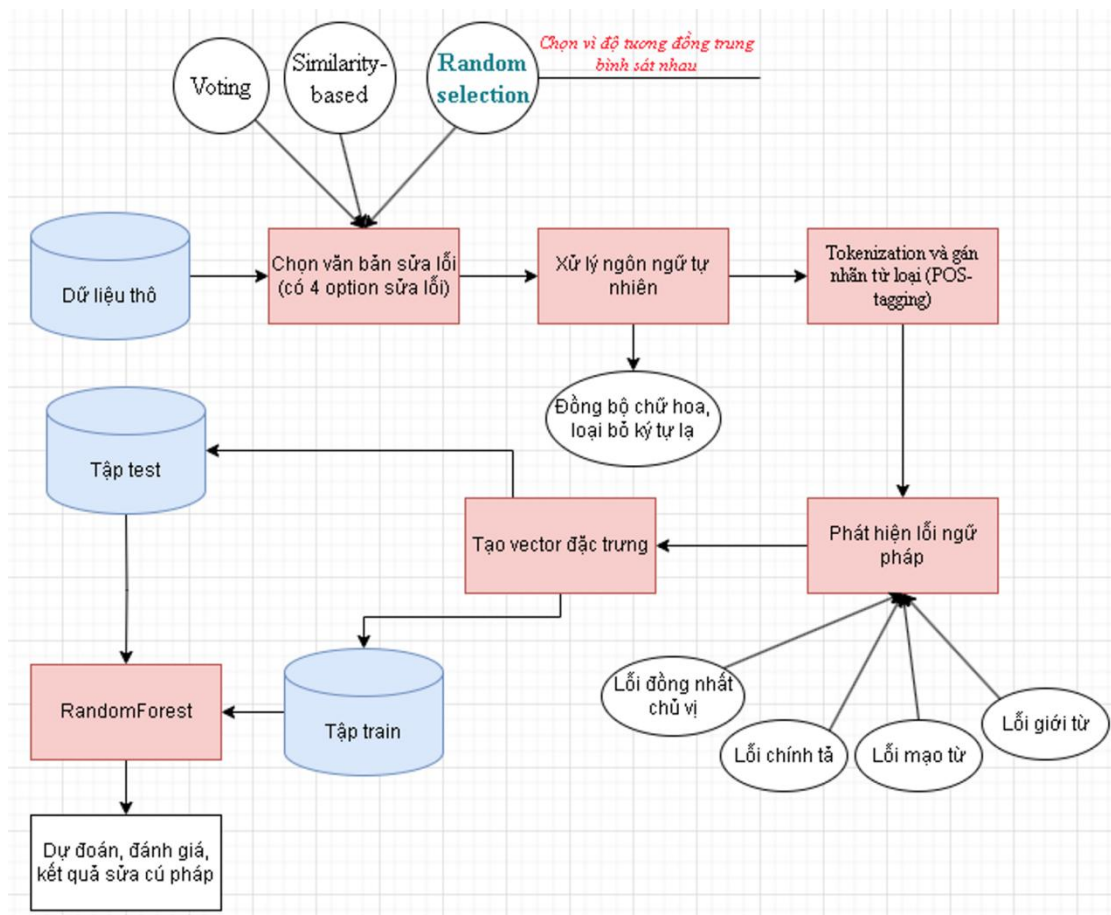
+ n\_estimator=300

+ max\_depth=30

+ random\_state=42

+ n\_jobs=-1

i) **Pipeline**



## 2. Mô hình T5 Transformer được fine-tune cho nhiệm vụ sửa lỗi ngữ pháp (Grammar Error Correction)

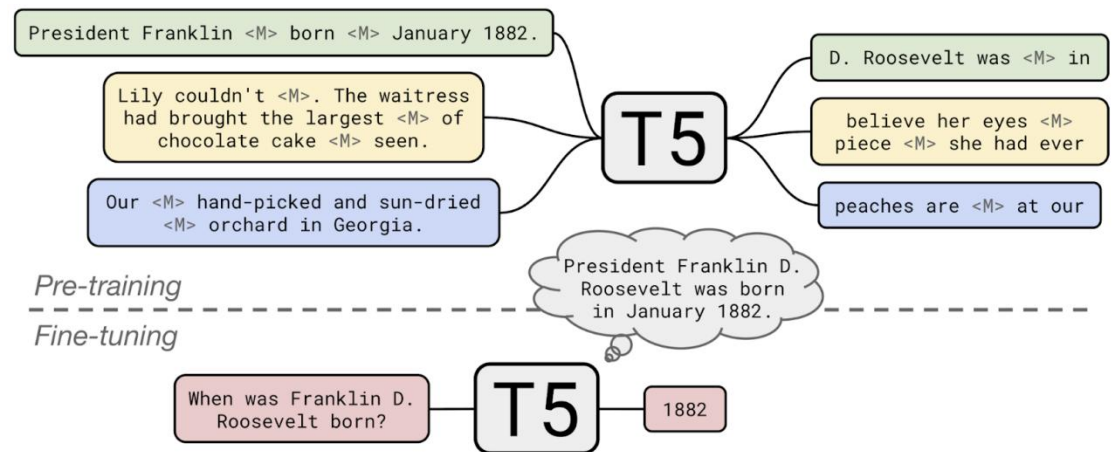
### a) Mô hình cơ bản:

#### ***T5 (Text-to-Text Transfer Transformer)***

Sử dụng pre-trained model "t5-small"

Fine-tuning trên JFLEG dataset

T5 (Text-to-Text Transfer Transformer) là một mô hình transformer dạng sinh (generative) được phát triển bởi Google Research (2020), với mục tiêu hợp nhất tất cả các tác vụ NLP dưới dạng một bài toán sinh văn bản (text-to-text).



Cách hoạt động:

- Mọi đầu vào và đầu ra đều là chuỗi văn bản.
- T5 sẽ học chuyển đổi từ văn bản đầu vào sang văn bản đầu ra, bất kể đó là:
  - Dịch ngôn ngữ
  - Tóm tắt văn bản
  - Trả lời câu hỏi
  - Phân loại
  - Sửa lỗi chính tả, ngữ pháp
  - Sinh văn bản sáng tạo

Kiến trúc:

Dựa hoàn toàn trên mô hình Transformer encoder-decoder (giống như seq2seq attention).

Mô hình sinh đầu ra theo từng token, dùng beam search hoặc greedy decoding.

Không có thành phần đặc biệt ngoài việc huấn luyện thống nhất cho nhiều tác vụ.

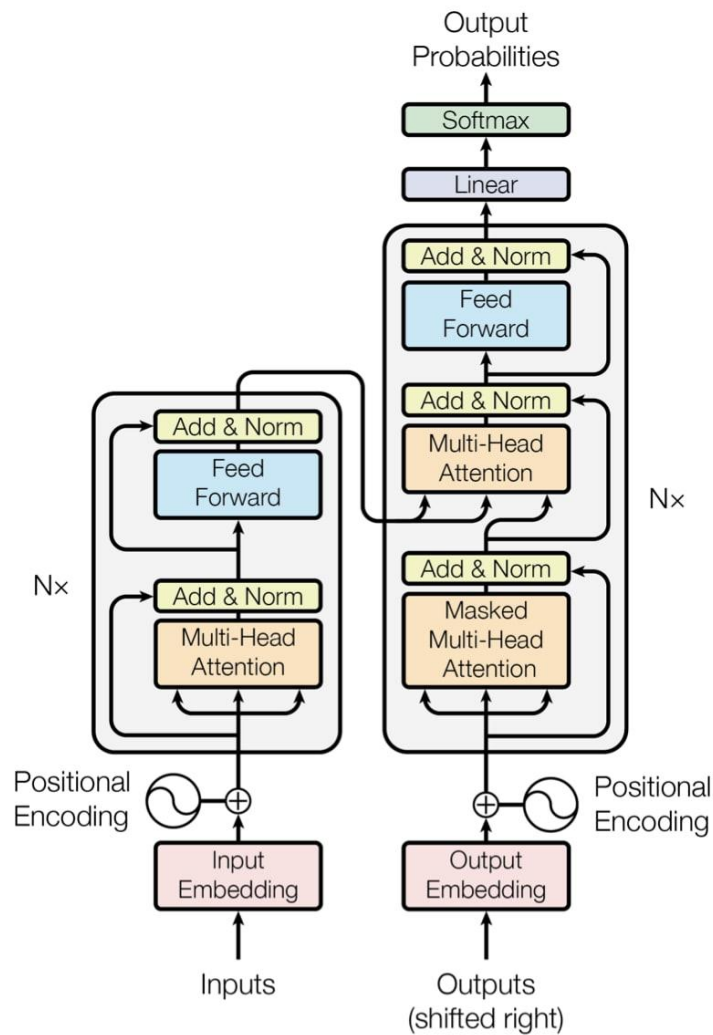


Figure 1: The Transformer - model architecture.

#### Ưu điểm:

Kiến trúc thống nhất  $\rightarrow$  dễ triển khai cho nhiều tác vụ  
 Hiệu quả cao, linh hoạt  
 Có nhiều checkpoint pretrained, dễ fine-tune  
 Hỗ trợ đa ngôn ngữ (mT5)

#### Nhược điểm

Cần nhiều tài nguyên tính toán (đặc biệt với mô hình lớn)  
 Tốc độ chậm khi dùng CPU (nên dùng GPU nếu có)  
 Nếu fine-tune không đủ tốt, dễ sinh ra câu sai ngữ nghĩa

#### Lý thuyết nền tảng:

#### + *Transformer Architecture (chi tiết encoder/decoder):*

| Thành phần | Mô tả                                            |
|------------|--------------------------------------------------|
| ◆ Encoder  | Nhận đầu vào và mã hóa nó thành vector đặc trưng |
| ◆ Decoder  | Dùng vector từ encoder để tạo ra đầu ra          |

Mỗi phần bao gồm N lớp giống nhau (thường là 6), mỗi lớp bao gồm các thành phần:

Multi-Head Attention

Feed Forward Neural Network (FFN)

Residual connection + Layer Normalization

### **ENCODER (Bộ mã hóa)**

#### 1. Input Embedding + Positional Encoding

Mã hóa từ (word/token) thành vector bằng embedding.

Thêm thông tin thứ tự vào bằng positional encoding (vì Transformer không có kiến trúc tuần tự như RNN/LSTM).

#### 2. N layers identical Encoders (thường là 6)

Mỗi lớp encoder gồm:

##### A. Multi-Head Self-Attention

Đầu vào tự tương tác với chính nó.

Cho phép mô hình học được mối quan hệ giữa các từ ở các vị trí khác nhau.

##### B. Add & Norm

Thêm đầu vào ban đầu với đầu ra attention → Chuẩn hóa lớp (LayerNorm)

##### C. Feed Forward Neural Network (FFN)

2 lớp fully connected

##### D. Add & Norm

Thêm đầu vào FFN + Normalize

### **DECODER (Bộ giải mã)**

Giống Encoder, nhưng thêm attention kết nối với encoder.

#### 1. Masked Multi-Head Self-Attention

Không cho phép mô hình nhìn thấy tương lai (tức là che các từ phía sau).

Masked bằng cách gán  $-\infty$  để softmax = 0.

#### 2. Add & Norm

#### 3. Multi-Head Attention với Encoder Output

Query đến từ decoder

Key và Value đến từ encoder output

→ Cho phép decoder "chú ý" tới các token đã mã hóa.

#### 4. Add & Norm

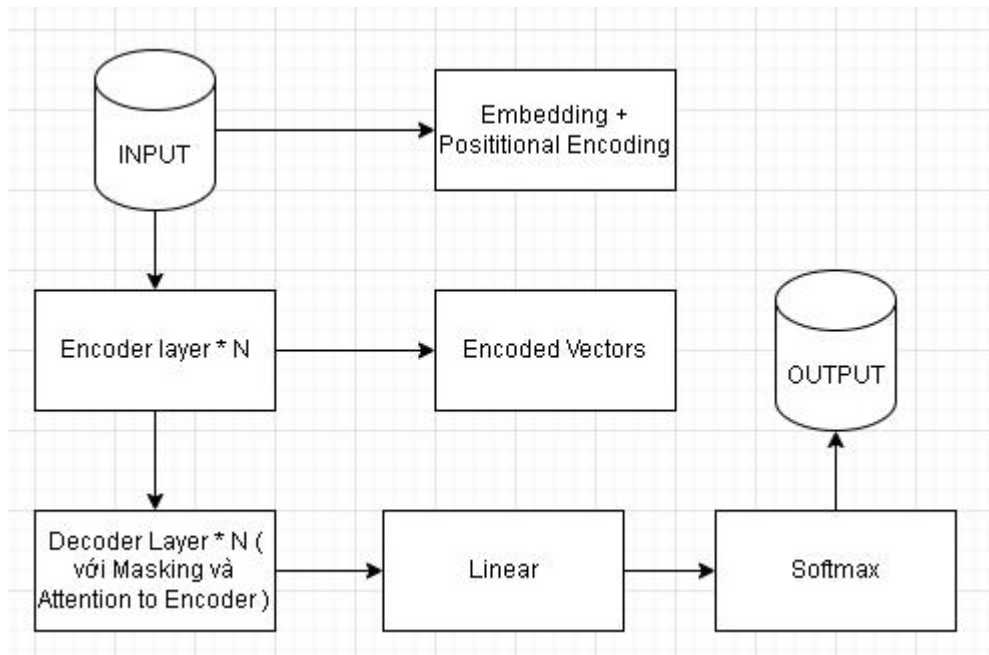
#### 5. Feed Forward Network (FFN)

#### 6. Add & Norm

#### 7. Final Linear + Softmax

Biến đầu ra decoder thành xác suất từ vựng để chọn từ tiếp theo.





**Attention Mechanism (Scaled Dot-Product, Multi-Head):** Cơ chế Attention là trái tim của kiến trúc Transformer, giúp mô hình "tập trung" vào các phần quan trọng trong chuỗi đầu vào.

### Scaled Dot-Product Attention (Chú ý điểm tích có tỉ lệ)

Ý tưởng: Cho 3 ma trận:

Q (Query)

K (Key)

V (Value)

Mục tiêu:

Tính attention-weighted sum của các vector V, trong đó trọng số đến từ mức độ tương đồng giữa Q và K.

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

**Giải thích từng phần:**

| Thành phần       | Mô tả                                              |
|------------------|----------------------------------------------------|
| $QK^T$           | Tính độ tương đồng giữa Query và Key (dot product) |
| $\sqrt{d_k}$     | Chia để tránh giá trị quá lớn → tránh gradient nhỏ |
| $\text{softmax}$ | Biến thành xác suất                                |
| V                | Dữ liệu sẽ được trộn theo trọng số vừa tính        |

Ví dụ trực quan:

Giả sử từ "ate" cần tập trung vào từ "she" trong câu "She ate the apple"

→ Attention sẽ tự động gán trọng số cao cho vector của "she".

### Multi-Head Attention (Chú ý đa đầu)

Vấn đề của attention đơn:

Một "đầu" attention chỉ học được một kiểu quan hệ giữa các từ (ví dụ: từ gần, từ lặp, từ nguyên nhân).

Giải pháp:

Tạo nhiều "đầu" attention song song → mỗi đầu học 1 quan hệ khác nhau.

Các bước thực hiện Multi-Head:

1. Chiếu Q,K,V thành nhiều phiên bản nhỏ:

$$Q_i = QW_i^Q, \quad K_i = KW_i^K, \quad V_i = VW_i^V$$

2. Tính attention cho từng **head**:

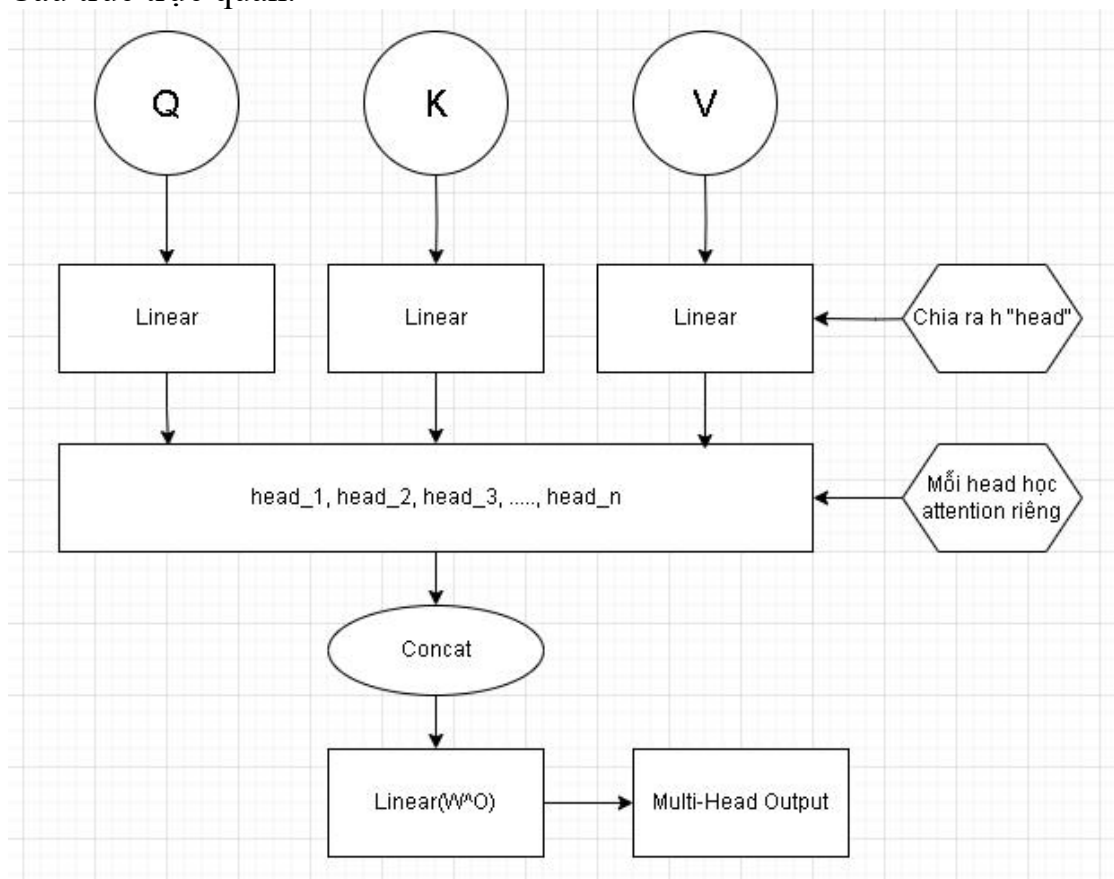
$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

3. Nối tất cả các đầu:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

| Bước                  | Mô tả                                  |
|-----------------------|----------------------------------------|
| 1. Linear projection  | Tách không gian thành nhiều hướng nhìn |
| 2. Scaled Dot-Product | Tính attention riêng biệt              |
| 3. Concat + Linear    | Gộp lại & học kết hợp các hướng nhìn   |

Cấu trúc trực quan:



## ***Positional Encoding:***

### **TẠI SAO CẦN POSITIONAL ENCODING?**

Transformer xử lý toàn bộ chuỗi cùng lúc, nên nó không biết từ nào đứng trước/sau nếu không có tín hiệu vị trí.

→ Vì thế, ta cộng thêm vector vị trí (PE) vào mỗi vector embedding đầu vào.

### **CÔNG THỨC TOÁN HỌC**

Cho:

pos: vị trí của từ (0, 1, 2, ...)

i: chiều thứ iii trong embedding

$D_{\text{model}}$  : kích thước vector (thường là 512)

Công thức:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{D_{\text{model}}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{D_{\text{model}}}}}\right)$$

Diễn giải:

Tại mỗi vị trí pospospos, ta tạo một vector PE có độ dài  $d_{\text{model}}$

Vector này được tính bằng cách áp dụng sine cho các chiều chẵn và cosine cho các chiều lẻ.

Mỗi chiều có một tần số riêng biệt → cho phép mô hình học được mối quan hệ vị trí.

### **PE ĐƯỢC DÙNG NHƯ THẾ NÀO?**

Khi tính embedding đầu vào cho một từ:

$$\text{Input} = \text{WordEmbedding} + \text{PositionalEncoding}$$

→ Giúp mô hình có thông tin về cả nội dung và vị trí.

### **TÍNH CHẤT HAY CỦA PE**

| Tính chất         | Giải thích                                                                    |
|-------------------|-------------------------------------------------------------------------------|
| Tuyến tính vị trí | Mối quan hệ giữa vị trí $pos$ và $pos + k$ có thể được mô hình hóa tuyến tính |
| Không cần học     | Là hàm xác định, không cần huấn luyện (train)                                 |
| Có thể khái quát  | Cho phép mô hình tổng quát sang độ dài chuỗi mới mà nó chưa từng thấy         |

## **b) Training setup:**

**Gradient checkpointing** (để tiết kiệm bộ nhớ): là một kỹ thuật tối ưu hóa bộ nhớ RAM/VRAM trong quá trình huấn luyện mô hình deep learning. Thay vì lưu toàn bộ các tensor trung gian (activations) trong quá trình lan truyền tiến (forward), kỹ thuật này chỉ lưu một số mốc

checkpoint và sẽ tính lại các phần còn lại khi lan truyền ngược (backward).

Bình thường:

Trong quá trình forward, mô hình lưu tất cả các activation để sử dụng trong backward.

Điều này tốn nhiều bộ nhớ, nhất là với mô hình nhiều lớp hoặc batch size lớn.

Với checkpointing:

Mô hình chỉ lưu các điểm mốc (checkpoints) ví dụ: sau mỗi vài layer.

Trong backward, nếu cần các activation khác → tính lại forward từ checkpoint gần nhất đến điểm đó.

Kết luận: Đổi bộ nhớ lấy tính toán → tiết kiệm RAM/VRAM nhưng chậm hơn một chút do tính lại.

***Chỉ số mô hình:***

Mixed precision training (FP16)

Batch size: 4

Gradient accumulation steps: 2

Training epochs: 3

Lưu checkpoint mỗi 300 steps

Logging mỗi 100 steps

**c) Generation parameters:**

***Max new tokens: 30***

***Beam search với beam size = 2***

***No sampling (deterministic)***

***Early stopping:*** Early stopping là một kỹ thuật chống overfitting trong quá trình huấn luyện mô hình học máy hoặc học sâu. Nó dừng quá trình huấn luyện sớm hơn dự kiến nếu mô hình không cải thiện thêm trên tập validation sau một số epoch nhất định.

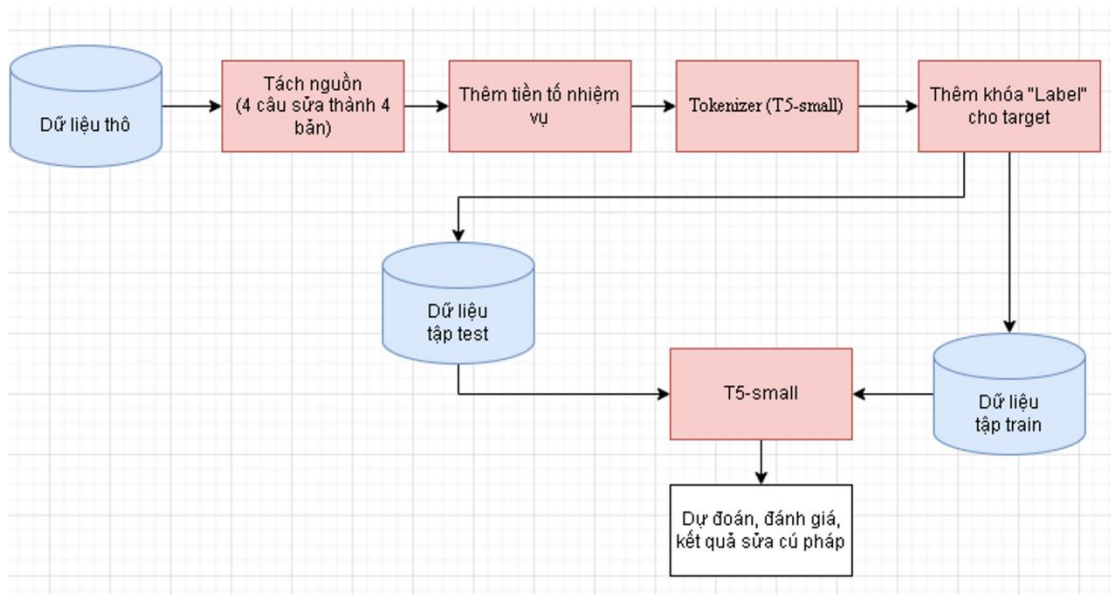
**d) Evaluation:**

***Sử dụng SacreBLEU để đánh giá***

***Corpus BLEU score trên test set***

***Multiple references evaluation***

**e) Model pipeline:**



### C. Các bước thực hiện

#### 1. Rule-based Random Forest GEC Model

##### - Chọn bản sửa lỗi (Correction Selection)

em định nghĩa 3 phương pháp lấy ra một câu sửa duy nhất trong dataset

(1) Random selection – chọn ngẫu nhiên một bản sửa; (2) Voting – chọn bản sửa xuất hiện nhiều nhất (dùng Counter); (3) Similarity-based – chọn bản sửa có độ tương đồng trung bình cao nhất với các bản khác.

```

import random

def get_correction_methods(row):
    corrections = row['corrections']

    # Method 1: Random selection
    random_correction = random.choice(corrections)

    # Method 2: Voting (if there are duplicates)
    from collections import Counter
    correction_counts = Counter(corrections)
    most_common = correction_counts.most_common(1)[0][0]

    # Method 3: Similarity-based selection
    avg_similarities = []
    for i, corr1 in enumerate(corrections):
        total_sim = sum(similarity_ratio(corr1, corr2) for j, corr2
in enumerate(corrections) if i != j)
        avg_sim = total_sim / (len(corrections) - 1)
        avg_similarities.append((avg_sim, corr1))
    most_similar = max(avg_similarities, key=lambda x: x[0])[1]

    return {
        'random': random_correction,
        'voting': most_common,
        'similarity': most_similar
    }

```

Nhưng vì khi thực hiện tính toán độ tương đồng trung bình của dataset em thấy rằng kết quả cho biết là các câu sửa trên có độ tương đồng khá giống nhau.

```

Sentence 0:
Original: So I think we can not live if old people could not find siences and tecnologies and they did not developped
Average similarity between corrections: 0.73

Sentence 1:
Original: For not use car .
Average similarity between corrections: 0.66

Sentence 2:
Original: Here was no promise of morning except that we looked up through the trees we saw how low the forest had swun
Average similarity between corrections: 0.95

Sentence 3:
Original: Thus even today sex is considered as the least important topic in many parts of India .
Average similarity between corrections: 0.98

Sentence 4:
Original: image you salf you are wark in factory just to do one thing like pot taire on car if they fire you you will
Average similarity between corrections: 0.62

```

Vì tính đơn giản nên em quyết định chọn cách là random selection - chọn bản sửa ngẫu nhiên Cụ thể, hàm `apply_random_correction` tạo hai cột `original` và `corrected` trong DataFrame bằng cách lấy ngẫu nhiên một bản sửa cho mỗi câu gốc.

```
# Áp dụng random selection cho toàn bộ dataset
def apply_random_correction(df):
    # Tạo copy của DataFrame để không ảnh hưởng đến dữ liệu gốc
    result_df = pd.DataFrame()
    result_df['original'] = df['sentence']
    result_df['corrected'] = df['corrections'].apply(lambda x:
random.choice(x))
    return result_df

# Áp dụng cho training và test set
train_processed = apply_random_correction(train_texts)
test_processed = apply_random_correction(test_texts)
```

### - Xử lý ngôn ngữ tự nhiên (Preprocessing)

Trước khi phát hiện lỗi, câu được chuẩn hóa và tách thành thành phần nhỏ. em dùng NLTK: chuyển chữ về chữ thường, loại bỏ ký tự lạ, sau đó tokenization và gán nhãn từ loại (POS-tagging)

```
Example 2:
Original: For not use car .
Corrected: Do not use in the car .

Features comparison:
Original POS tags: [('for', 'IN'), ('not', 'RB'), ('use', 'JJ'), ('car', 'NN'), ('.', '.')]
Corrected POS tags: [('do', 'VB'), ('not', 'RB'), ('use', 'VB'), ('in', 'IN'), ('the', 'DT'), ('car', 'NN'), ('.', '.')]

```

Quá trình này cho phép phát hiện các lỗi dựa trên ngữ pháp (như đồng nhất chủ-vị) và ngữ cảnh từ loại. Ngoài ra, em còn trích xuất các n-gram (2-gram, 3-gram) phục vụ mục đích phân tích ngữ cảnh.

### - Phát hiện lỗi ngữ pháp (Error Detection)

```

# Hàm tiền xử lý cơ bản
def preprocess_text(text):
    # Lowercase
    text = text.lower()
    # Chuẩn hóa dấu câu
    text = re.sub(r'^\w\s.,!?', '', text)
    # Tokenize
    tokens = word_tokenize(text)
    # POS tagging
    pos_tags = pos_tag(tokens)
    # N-grams (2-gram và 3-gram)
    bigrams = list(ngrams(tokens, 2))
    trigrams = list(ngrams(tokens, 3))

    return {
        'tokens': tokens,
        'pos_tags': pos_tags,
        'bigrams': bigrams,
        'trigrams': trigrams
    }

```

em triển khai các hàm để phát hiện bốn loại lỗi chính:

- Lỗi chính tả (spelling): Sử dụng thư viện SpellChecker để nhận diện từ không có trong từ điển. Nếu một từ (không phải từ viết hoa hay quá ngắn) không có trong bộ từ điển, hàm sẽ đề xuất tập ứng viên sửa lỗi theo khoảng cách Levenshtein. Ưu điểm của phương pháp này là đơn giản và sử dụng nguồn dữ liệu có sẵn, nhưng nhược điểm là bị hạn chế bởi từ điển (có thể bỏ sót từ mới) và các gợi ý có thể không phù hợp ngữ cảnh.



```
def detect_spelling_errors(tokens):
    misspelled = []
    for i, token in enumerate(tokens):
        # Bỏ qua các từ viết hoa và từ ngắn
        if token.isupper() or len(token) < 2:
            continue
        # Kiểm tra lỗi chính tả
        if token.lower() not in spell:
            try:
                candidates = list(spell.candidates(token))
            except:
                candidates = [] # Nếu không tìm được suggestions thì
# để rỗng
                misspelled.append({
                    'position': i,
                    'word': token,
                    'suggestions': candidates if candidates else []
                })
    return misspelled
```

- Lỗi mạo từ (article): Hàm `detect_article_errors` kiểm tra cặp hai từ liên tiếp: nếu một từ là “a” hoặc “an” và từ tiếp theo là danh từ, sẽ xác định xem nên dùng “a” hay “an” dựa trên nguyên âm ở đầu từ. Ví dụ, nếu “a” đi trước một từ bắt đầu bằng nguyên âm, hàm gợi ý chuyển thành “an” và ngược lại. Phương pháp này dựa trên quy tắc đơn giản về nguyên âm, dễ triển khai nhưng không xử lý hết trường hợp phức

```
def detect_article_errors(pos_tags):
    article_errors = []
    for i in range(len(pos_tags)-1):
        word, tag = pos_tags[i]
        next_word, next_tag = pos_tags[i+1]

        if word.lower() in ['a', 'an']:
            if next_tag.startswith('NN'):
                should_be_an = next_word[0].lower() in 'aeiou'
                if (word.lower() == 'a' and should_be_an) or
(word.lower() == 'an' and not should_be_an):
                    article_errors.append((i, word, 'an' if
should_be_an else 'a'))
    return article_errors
```

tạp về ngữ âm hay danh từ trừu tượng.

- Lỗi giới từ (preposition): Sử dụng một từ điển các giới từ thông dụng để gợi ý thay đổi. Ví dụ, nếu từ loại là giới từ (“IN” trong POS-tag) và từ đó nằm trong danh sách (ví dụ “in”, “on”, “at”, “to”, “for”...),

hàm `detect_preposition_errors` trả về các gợi ý khác để thay thế. Đây là quy tắc rất đơn giản, ưu điểm là nhanh chóng, nhưng nhược điểm là

```
COMMON_PREPOSITIONS = {
    'in': ['at', 'on', 'by'],
    'on': ['in', 'at', 'by'],
    'at': ['in', 'on', 'by'],
    'to': ['for', 'into', 'toward'],
    'for': ['to', 'of', 'with'],
    'with': ['by', 'in', 'through'],
    'of': ['from', 'about', 'with'],
    'by': ['with', 'through', 'from']
}

def detect_preposition_errors(pos_tags):
    preposition_errors = []
    for i, (word, tag) in enumerate(pos_tags):
        if tag == 'IN' and word.lower() in COMMON_PREPOSITIONS:
            preposition_errors.append({
                'position': i,
                'word': word,
                'suggestions': COMMON_PREPOSITIONS[word.lower()]
            })
    return preposition_errors
```

thiếu ngữ cảnh (có thể không phù hợp ngữ nghĩa) và danh sách có thể không đầy đủ.

- Lỗi đồng nhất chủ-vị (subject-verb agreement): Dựa trên POS-tag, hàm `detect_subject_verb_agreement` kiểm tra các cặp (chủ ngữ, động từ) đi liền nhau. Chẳng hạn chủ ngữ “I/you/we/they” thường đi với động từ dạng không chia (base form), còn “he/she/it” đi với dạng chia thêm “-s” (VBZ). Nếu phát hiện vi phạm (ví dụ “She go” hay “I goes”), hàm ghi nhận lỗi và đề xuất dạng đúng hơn (ví dụ chuyển “go”->“goes”). Cách làm này dùng quy tắc chuẩn về danh từ và động từ, ưu điểm là trực tiếp, nhưng nhược điểm là không phát hiện hết các trường hợp phức tạp (ví dụ động từ bất quy tắc hay câu dài hơn).

```

def detect_subject_verb_agreement(pos_tags):
    agreement_errors = []
    subject_verb_pairs = [
        # I/you/we/they + base form
        (['PRP'], ['VBZ']),
        # he/she/it + 3rd person
        (['PRP'], ['VBP']),
        # Singular noun + base form
        (['NN'], ['VBP']),
        # Plural noun + 3rd person
        (['NNS'], ['VBZ'])
    ]

    for i in range(len(pos_tags)-1):
        word, tag = pos_tags[i]
        next_word, next_tag = pos_tags[i+1]

        # Check common subject-verb agreement patterns
        for subj_tags, verb_tags in subject_verb_pairs:
            if tag in subj_tags and next_tag in verb_tags:
                if word.lower() in ['i', 'you', 'we', 'they'] and next_tag ==
'VBZ':
                    agreement_errors.append({
                        'position': i+1,
                        'word': next_word,
                        'type': 'subject-verb agreement',
                        'context': f"{word} {next_word}",
                        'suggestion': 'use base form'
                    })
                elif word.lower() in ['he', 'she', 'it'] and next_tag == 'VBP':
                    agreement_errors.append({
                        'position': i+1,
                        'word': next_word,
                        'type': 'subject-verb agreement',
                        'context': f"{word} {next_word}",
                        'suggestion': 'use third person singular'
                    })
    return agreement_errors

```

Kết quả của bước phát hiện lỗi được minh hoạ qua ví dụ: với câu “So I think we can not live if old people could not find siences...”, hàm phát hiện 3 lỗi chính tả và gợi ý sửa (“siences”→“sciences”, “tecnologies”→“technologies”, “developped”→“developed”). Với câu “For not use car.”, hàm phát hiện lỗi giới từ (gợi ý “for”→“to”).

```
Original: So I think we can not live if old people could not  
Corrected: So I think we can not live if old people could not  
  
Detected errors:  
  
Spelling Errors:  
Word 'siences' at position 13  
Suggestions: ['sciences', 'silences']  
Word 'tecnologies' at position 15  
Suggestions: ['technologies']  
Word 'developped' at position 20  
Suggestions: ['developed']
```

### - Tạo vector đặc trưng và huấn luyện mô hình (Feature Engineering & Modeling)

```
features = {  
    'has_spelling_errors':  
bool(orig_errors['spelling_errors']),  
    'spelling_error_count':  
len(orig_errors['spelling_errors']),  
    'has_article_errors':  
bool(orig_errors['article_errors']),  
    'article_error_count':  
len(orig_errors['article_errors']),  
    'has_preposition_errors':  
bool(orig_errors['preposition_errors']),  
    'preposition_error_count':  
len(orig_errors['preposition_errors']),  
    'has_agreement_errors':  
bool(orig_errors['agreement_errors']),  
    'agreement_error_count':  
len(orig_errors['agreement_errors']),  
    'text_length': len(text.split()),  
    'avg_word_length': sum(len(word) for word in  
text.split()) / len(text.split()) if text else 0  
}
```

| Tên đặc trưng                        | Kiểu dữ liệu       | Ý nghĩa                                                                                        |
|--------------------------------------|--------------------|------------------------------------------------------------------------------------------------|
| <code>has_spelling_errors</code>     | <code>bool</code>  | Câu có lỗi chính tả hay không. Giúp mô hình nhận biết câu có lỗi dạng gõ sai từ.               |
| <code>spelling_error_count</code>    | <code>int</code>   | Số lỗi chính tả trong câu. Đánh giá mức độ nghiêm trọng của lỗi.                               |
| <code>has_article_errors</code>      | <code>bool</code>  | Câu có lỗi mạo từ (a/an/the) hay không. Phổ biến trong tiếng Anh, giúp nhận diện lỗi ngữ pháp. |
| <code>article_error_count</code>     | <code>int</code>   | Số lỗi mạo từ. Cho biết người viết có dùng sai mạo từ thường xuyên không.                      |
| <code>has_preposition_errors</code>  | <code>bool</code>  | Câu có lỗi giới từ hay không (in/on/at/by...). Quan trọng trong viết học thuật.                |
| <code>preposition_error_count</code> | <code>int</code>   | Số lượng lỗi giới từ.                                                                          |
| <code>has_agreement_errors</code>    | <code>bool</code>  | Câu có lỗi hòa hợp chủ vị (Subject-Verb Agreement) không. Rất thường gặp ở người học tiếng.    |
| <code>agreement_error_count</code>   | <code>int</code>   | Số lượng lỗi hòa hợp chủ vị.                                                                   |
| <code>text_length</code>             | <code>int</code>   | Độ dài câu tính theo số từ (token). Câu dài có thể dễ mắc lỗi hơn.                             |
| <code>avg_word_length</code>         | <code>float</code> | Độ dài trung bình mỗi từ. Có thể phản ánh độ phức                                              |

Sau khi phát hiện lỗi, em xây dựng vector đặc trưng cho từng câu gốc. Ban đầu, hàm `create_feature_vector` tạo các tính năng đơn giản từ kết quả phát hiện lỗi: có lỗi chính tả hay không, số lỗi chính tả, có lỗi mạo từ, lỗi giới từ, lỗi đồng nhất, chiều dài câu, độ dài trung bình của từ, v.v. Ngoài ra còn bổ sung chuỗi POS của cả câu và đếm số động từ. Label (`needs_correction`) được xác định là True nếu câu gốc khác câu sửa hoặc có bất kỳ lỗi nào. Để cải thiện, em định nghĩa thêm `create_advanced_feature_vector` với các tính năng nâng cao: tỉ lệ lỗi trên số từ (`spelling/error ratio`), tỉ lệ các loại từ (động từ, danh từ, tính từ) theo

```
# Thêm POS sequence features
pos_sequence = ' '.join([tag for word, tag in pos_tags])
features['pos_sequence'] = pos_sequence

# Thêm verb agreement features
verb_tags = ['VB', 'VBD', 'VBG', 'VBN', 'VBP', 'VBZ']
features['verb_count'] = sum(1 for _, tag in pos_tags if tag
in verb_tags)
```

POS, số từ trong câu, v.v. Các feature này giúp mô hình nắm bắt cấu trúc và tần suất lỗi tốt hơn.

Mô hình được chọn là Random Forest (một ensemble của nhiều cây quyết định), được triển khai qua RandomForestClassifier trong Scikit-learn. Lý do chọn Random Forest bao gồm: nó xử lý tốt dữ liệu có nhiều feature hỗn hợp, ít yêu cầu tiền xử lý, giảm overfitting nhờ tính chất ensemble, và cho khả năng tính toán feature quan trọng. Tuy nhiên, nhược điểm là khó diễn giải trực tiếp và có thể đòi hỏi nhiều tài nguyên tính toán. em chia dữ liệu thành tập huấn luyện và tập validation (sử dụng train\_test\_split), huấn luyện mô hình với pipeline gồm StandardScaler (chuẩn hóa dữ liệu) và Random Forest, sau đó đánh giá trên tập kiểm thử.

```
Analyzing: I has a dog
Correct form: I have a dog
Detected as needing correction: True
Features found:
- text_length: 4
- avg_word_length: 2.0
- pos_sequence: NN VBZ DT NN
- verb_count: 1
- has_potential_agreement_error: True

Analyzing: She go to school everyday
Correct form: She goes to school everyday
Detected as needing correction: True
Features found:
- has_agreement_errors: True
- agreement_error_count: 1
- text_length: 5
- avg_word_length: 4.2
- pos_sequence: PRP VBP TO NN NN
- verb_count: 1
```

## - Hoạt động

Gán nhãn (label)

Với mô hình binary: 0 = không lỗi, 1 = có lỗi

Với mô hình đa lớp: 0 = đúng, 1 = lỗi chính tả, 2 = lỗi mạo từ, 3 = lỗi giới từ,...

Random Forest:

Huấn luyện nhiều cây quyết định (decision trees) khác nhau trên các tập con ngẫu nhiên của dữ liệu và đặc trưng.

Mỗi cây sẽ học cách phân tách các đặc trưng như:

Nếu has\_article\_error=True và avg\_word\_length < 4 thì → Có lỗi

Nếu spelling\_error\_count > 2 thì → Có lỗi chính tả

Output:

Mỗi cây dự đoán nhãn (loại lỗi), và toàn bộ rừng bỏ phiếu (voting) để chọn kết quả cuối cùng.

Nhờ vào các đặc trưng mà random forest có thể nhận biết và tìm được chỗ nào cần sửa, vì sao sai. Và việc sửa lỗi sẽ nhờ vào Rule-base.

Rule-base là 1 luật cú pháp tiếng anh. Ví dụ:

- Danh từ số ít -> phải đi với động từ được chia ( thêm s như: he goes to school)

- Lỗi mạo từ: in the table -> on the table

- .....

### Nói tóm lại:

Random Forest chịu trách nhiệm phát hiện vị trí sai

Rule-based system chịu trách nhiệm sửa lỗi tại các vị trí đó

### - Kết quả và đánh giá (Results and Discussion)

| Validation Metrics: |           |        |          |         |
|---------------------|-----------|--------|----------|---------|
|                     | precision | recall | f1-score | support |
| False               | 0.25      | 0.12   | 0.17     | 8       |
| True                | 0.95      | 0.98   | 0.97     | 143     |
| accuracy            |           |        | 0.93     | 151     |
| macro avg           | 0.60      | 0.55   | 0.57     | 151     |
| weighted avg        | 0.92      | 0.93   | 0.92     | 151     |

Precision (độ chính xác):

False: 0.25 (25% dự đoán "không cần sửa" là đúng)

True: 0.95 (95% dự đoán "cần sửa" là đúng)

Recall (độ bao phủ):

False: 0.12 (12% câu thực sự không cần sửa được phát hiện)

True: 0.98 (98% câu thực sự cần sửa được phát hiện)

F1-score (trung bình điều hòa của precision và recall):

False: 0.17

True: 0.97

Support: Số lượng mẫu mỗi class

False: 8 câu không cần sửa

True: 143 câu cần sửa

Tổng: 151 câu

Metrics tổng hợp:

Accuracy: 0.93 (93% dự đoán đúng)

Macro avg: Trung bình cộng của các metrics (không quan tâm số lượng mẫu)

Precision: 0.60

Recall: 0.55

F1: 0.57

Weighted avg: Trung bình có trọng số (có tính đến số lượng mẫu)

Precision: 0.92

Recall: 0.93

F1: 0.92

```
Testing automatic error correction:

Original: She go to school everyday
Corrected: She goes to school everyday
Fixed errors:

Agreement Errors:
- Changed 'go' at position 1
  Applied: use third person singular

Original: He work in office
Corrected: He work at the office
Fixed errors:

Preposition Errors:
- Changed 'in' at position 2
```

```
Test Set Metrics:
Exact Match Accuracy: 0.1270
Correction F1-score: 0.8624
Average Levenshtein Distance: 10.5561
Corpus BLEU Score: 0.6644
Average Sentence BLEU Score: 0.6235
```

Kết quả trên là em chạy trên 1 tập test hoàn toàn mới với hơn 754 câu.  
Exact Match Accuracy (0.1270 ~ 12.7%)

Chỉ có khoảng 12.7% các câu được sửa hoàn toàn giống với ground truth

Con số này thấp vì một câu có thể có nhiều cách sửa đúng khác nhau

Đây không phải metric tốt nhất để đánh giá mô hình GEC  
Correction F1-score (0.8620 ~ 86.2%)

F1-score cao cho thấy mô hình có khả năng phát hiện và sửa lỗi tốt

Balance tốt giữa precision và recall

Mô hình ít bỏ sót lỗi và ít sửa sai những chỗ đúng

Average Levenshtein Distance (10.5561)

Trung bình cần 10-11 thao tác chỉnh sửa để biến câu gốc thành câu đích

Con số này hơi cao, cho thấy mô hình thường sửa nhiều từ trong câu

Có thể do mô hình đang sửa cả những chỗ không cần thiết

BLEU Scores

Corpus BLEU (0.6644 ~ 66.44%)



Average Sentence BLEU (0.6235 ~ 62.35%)

BLEU scores > 0.6 là khá tốt cho task GEC

Cho thấy các câu được sửa vẫn giữ được nhiều nội dung và cấu trúc của câu gốc

#### Ưu điểm của mô hình:

F1-score cao (86.2%) cho thấy độ chính xác tốt trong việc sửa lỗi

BLEU scores cao (>60%) chứng tỏ giữ được ngữ nghĩa của câu gốc

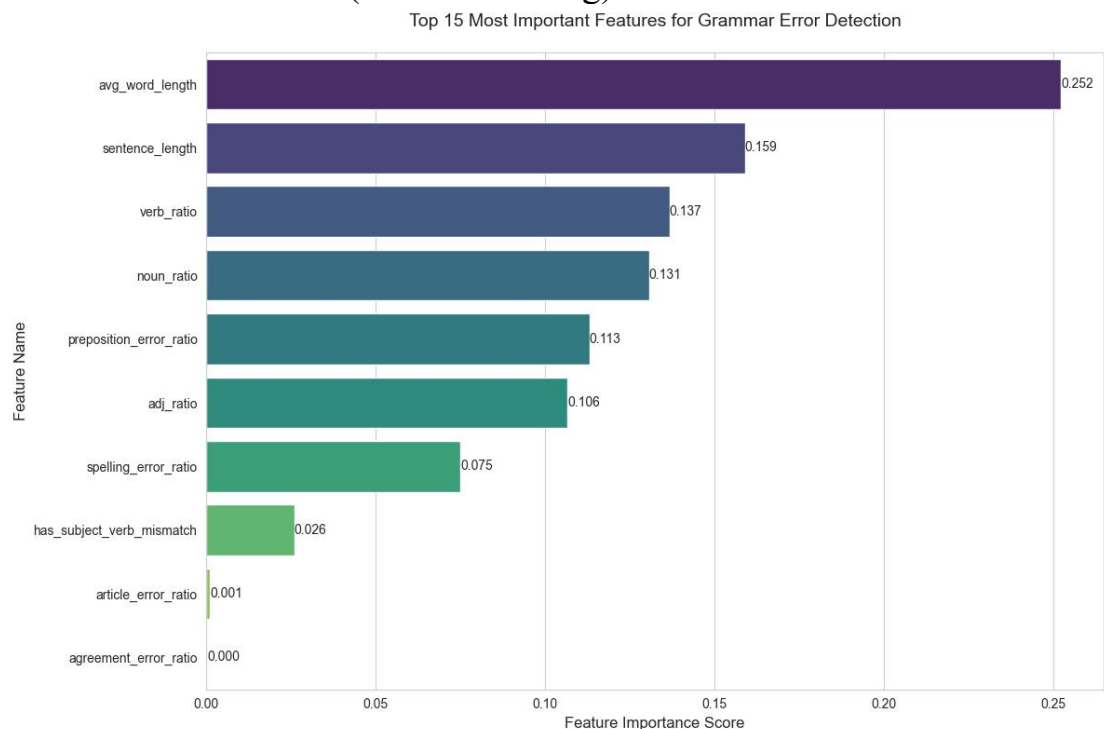
Có khả năng xử lý nhiều loại lỗi khác nhau

#### Nhược điểm:

Exact Match Accuracy thấp (12.7%)

Levenshtein Distance cao (10.5) cho thấy sửa nhiều từ

-> Có thể over-correct (sửa cả chỗ đúng)



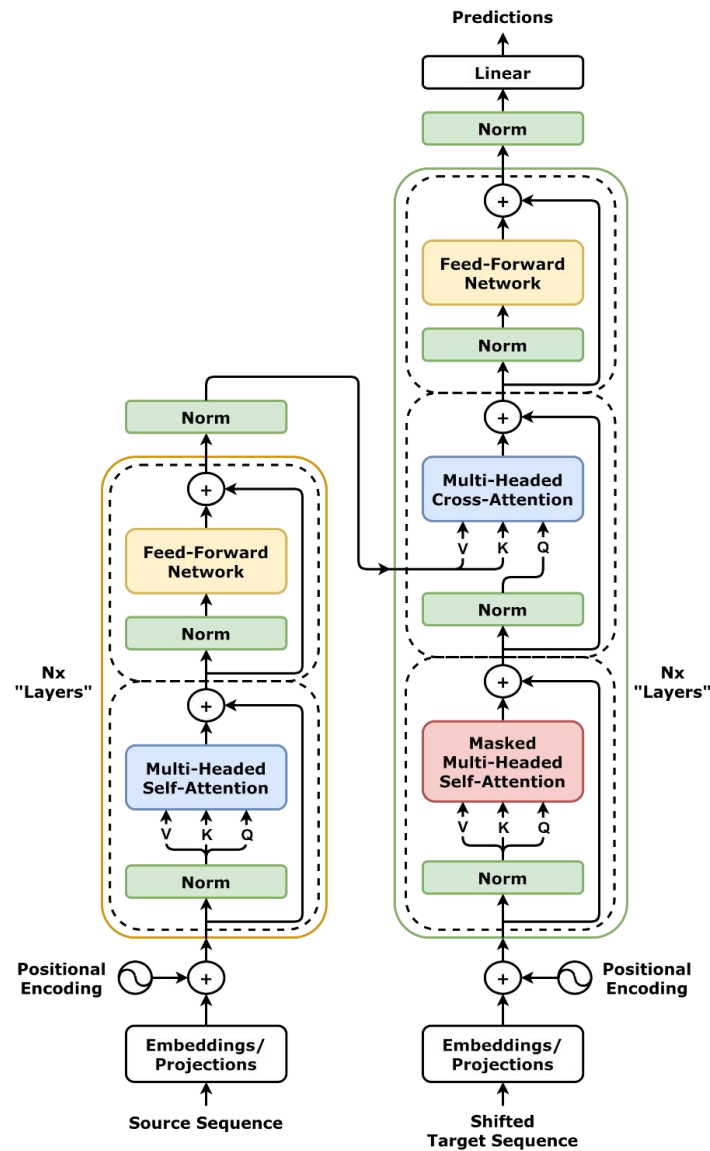
#### *Biểu đồ top đặc trưng quan trọng*

## 2. Mô hình T5 Transformer được fine-tune

Em sử dụng mô hình T5 của HuggingFace (T5ForConditionalGeneration), vốn là một **kiến trúc Transformer mã hóa – giải mã** (encoder-decoder) được huấn luyện trước và tinh chỉnh cho nhiệm vụ này[1]. Transformer là một kiến trúc học sâu không dùng RNN, dựa hoàn toàn vào cơ chế self-attention. Tổng quát, một Transformer bao gồm: lớp chuyển đổi từ đầu vào thành vector nhúng (token embedding), cộng với **mã hóa vị trí** (positional encoding) để nhận diện thứ tự từ[2]; sau đó có nhiều lớp Transformer xen kẽ giữa các khối attention và các mạng nơ-ron feed-forward[2]. Mô hình được kết thúc bằng một tầng unembedding (linear + softmax) cho phép chuyển vector cuối cùng thành xác suất cho từng token đầu ra.

Trong cơ chế **attention (chú ý)** của Transformer, mỗi token được biểu diễn bằng ba vector: query (truy vấn), key (khóa) và value (giá trị) thông qua ba ma trận trọng số học được ( $W^Q, W^K, W^V$ )[2]. Với một từ đầu vào, ta nhân vector nhúng với  $W^Q$  để được query, và tương tự với các từ khác để được ma trận key  $K$  và value  $V$ [2]. Điểm chú ý giữa hai từ  $i$  và  $j$  được tính bằng tích vô hướng giữa query của  $i$  và key của  $j$ , chia cho  $\sqrt{d_k}$  rồi qua hàm softmax để chuẩn hóa[2]. Các điểm này quyết định trọng số của từng value trong đầu ra. Cụ thể, đầu ra của khối attention cho vị trí  $i$  là tổng trọng số của tất cả value theo các điểm attention  $a_{ij}$ [2]. Transformer dùng cơ chế **Scaled Dot-Product Attention** với công thức phổ biến  $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$ [2]. Một phần quan trọng nữa là **Multi-Head Attention (chú ý nhiều đầu)**. Thay vì chỉ dùng một bộ query/key/value, mô hình tách thành nhiều “đầu” (heads), mỗi đầu có ma trận  $W^Q, W^K, W^V$  riêng. Các đầu này học cách chú ý theo các cách khác nhau cho mỗi token, giúp mô hình nắm bắt nhiều khía cạnh ngữ cảnh[2]. Sau đó kết quả của các đầu được ghép lại và chiếu qua một ma trận  $W^O$  để trả về kích thước ban đầu. Như vậy, cơ chế attention cho phép mọi token trong chuỗi đầu vào “tương tác” với nhau song song (không cần xử lý tuần tự), và các đầu khác nhau thì tập trung vào các phần khác nhau của ngữ cảnh.

**Mã hóa vị trí** (positional encoding) là cơ chế bổ sung thông tin về thứ tự các từ vì mô hình chỉ thấy vector nhưng không biết thứ tự ban đầu. Trong Transformer gốc, tác giả dùng hàm sin/cos đặc trưng để mã hóa chỉ số vị trí thành vector[2]. Nhờ đó, mô hình hiểu được ví dụ “man bites dog” khác với “dog bites man”, vì cùng nội dung từ mà thứ tự khác nhau.



Hình trên, Kiến trúc điển hình của mô hình Transformer (encoder-decoder) gồm nhiều lớp (lớp) stacked. Mỗi lớp encoder bao gồm khối **Multi-Head Self-Attention** và mạng **Feed-Forward** xen kẽ, kèm theo các kết nối dư (residual) và chuẩn hóa. Mỗi lớp decoder có thêm một khối **Masked Self-Attention** (tự chú ý có che) và một khối **Cross-Attention** nối với đầu ra encoder. Tầng đầu vào sử dụng **token embedding** cộng với mã hóa vị trí, tầng cuối cùng là un-embedding ra xác suất token đầu ra.

Trong hình trên, nửa trái (Encoder) xử lý toàn bộ câu nguồn (không cần che), cho phép mọi token chú ý đến nhau. Nửa phải (Decoder) gồm hai loại attention: một “Masked Self-Attention” chỉ cho phép token ở vị trí  $i$  tính chú ý dựa trên các token trước  $i$  (để tránh nhìn trước), và một “Encoder-Decoder Attention” (cross-attention) để lấy thông tin từ mã hóa đầu ra của encoder. Việc che (masking) ở decoder đảm bảo mô hình sinh ra câu theo kiểu tự hồi quy, không dùng thông tin của từ tương lai.

## - Tiền xử lý dữ liệu JFLEG

Trước khi huấn luyện, em thực hiện các bước tiền xử lý dữ liệu như sau: sau khi tải bộ JFLEG, mỗi dòng dữ liệu ban đầu có một câu nguồn (“sentence”) và một danh sách nhiều câu sửa (“corrections”). Mã Python sử dụng Pandas explode để tách mỗi câu nguồn ra thành nhiều bản, mỗi bản kèm một trong các câu sửa tương ứng. Kết quả là tập dữ liệu huấn luyện có kích thước lớn gấp khoảng bốn lần ban đầu (tương ứng với bốn câu sửa của mỗi câu nguồn).

Sau đó, notebook đổi tên cột “sentence” thành “input” và “corrections” thành “target”, và thêm tiền tố "grammar: " vào đầu mỗi câu đầu vào. Ví dụ, câu nguồn “She go to school everyday” sẽ thành đầu vào "grammar: She go to school everyday" và mục tiêu (target) là câu đã hiệu chỉnh “She goes to school every day”.

```
df = df.explode("corrections", ignore_index=True)
df.rename(columns={"sentence": "input", "corrections": "target"},
inplace=True)
df["input"] = "grammar: " + df["input"]
```

Việc thêm tiền tố “grammar:” theo khuyến nghị của mô hình T5 cho các nhiệm vụ khác nhau[1], giúp mô hình biết nhiệm vụ cần thực hiện. Cuối cùng, tập dữ liệu được chuyển sang định dạng HuggingFace Dataset (Dataset.from\_pandas) chứa hai trường “input” và “target”. Tiếp theo, mã định nghĩa một hàm tokenize sử dụng T5Tokenizer để ánh xạ mỗi câu sang chuỗi số (token IDs). Cụ thể, tokenizer mã hóa cả trường input và target với độ dài tối đa 128 token (padding/truncation). Trong output của hàm tokenize, các input\_ids của trường target được gán cho khóa "labels", để Trainer biết chúng là nhãn đích cho bài toán sinh văn bản. Kết quả cuối cùng là một dataset token đã sẵn sàng cho quá trình huấn luyện.

```
from datasets import Dataset
from transformers import T5Tokenizer

ds = Dataset.from_pandas(df[["input", "target"]])
tokenizer = T5Tokenizer.from_pretrained("t5-base")
def tokenize_fn(ex):
    enc = tokenizer(ex["input"], truncation=True,
padding="max_length", max_length=128)
    dec = tokenizer(ex["target"], truncation=True,
padding="max_length", max_length=128)
    enc["labels"] = dec["input_ids"]
    return enc
ds_tok = ds.map(tokenize_fn, batched=True, remove_columns=["input",
"target"])
```

## - Cấu trúc và huấn luyện mô hình

Em sử dụng mô hình **T5ForConditionalGeneration** (T5 nhỏ) từ thư viện Transformers của HuggingFace. T5 là một biến thể Transformer encoder-decoder, được huấn luyện trước (pre-trained) trên nhiều tác vụ text-to-text chung, và được gọi là “text-to-text transfer transformer”[1].

Mô hình này xử lý đầu vào là một chuỗi văn bản và sinh ra chuỗi văn bản đầu ra theo dạng ánh xạ. Trong code, mô hình được khởi tạo từ checkpoint "t5-small", và lập tức gọi `gradient_checkpointing_enable()` để tiết kiệm bộ nhớ GPU trong huấn luyện nặng (có thể hiểu là một kỹ thuật giảm bộ nhớ khi tính gradient).

Người dùng không thấy rõ định nghĩa loss vì HuggingFace Trainer sẽ sử dụng loss mặc định là cross-entropy (phù hợp với bài toán dịch/ngữ pháp).

Tóm tắt các thành phần:

Lớp embedding/token embedding: Biến mỗi token đầu vào thành vector (Token embedding) và cộng vector positional encoding[2].

Encoder: Gồm N lớp giống nhau, mỗi lớp có một khối Multi-Head Self-Attention và một mạng feed-forward, kèm kết nối dư và layer norm[2].

Decoder: Gồm N lớp, mỗi lớp có khối Masked Self-Attention (chỉ nhìn các token trước), khối Cross-Attention lấy các vector encode từ encoder, và mạng feed-forward[2]. Kết quả decoder đi qua tầng linear+softmax để dự đoán token kế tiếp.

Loss và optimizer: Trainer mặc định dùng cross-entropy giữa đầu ra sinh ra và nhãn “labels” (target tokens). Để tối ưu, mặc định Trainer dùng AdamW. Trong notebook, huấn luyện trong 3 epoch với batch nhỏ (4) và gradient accumulation để phù hợp với tài nguyên, cùng chế độ fp16 để tăng tốc trên GPU.

Trong quá trình huấn luyện, Trainer của HuggingFace tự động thực hiện quá trình lan truyền xuôi (forward) và ngược (backward) với loss, và lưu model.

Kết quả huấn luyện (theo bảng log) cho thấy loss huấn luyện giảm dần từ ~1.80 xuống ~0.13 (sau ~1100 bước) – minh chứng mô hình đã học được cách sinh câu đích khớp hơn với dữ liệu nhãn. Sau khi huấn luyện xong, mô hình và tokenizer được lưu để dùng tiếp.

```
model = T5ForConditionalGeneration.from_pretrained("t5-small")
model.gradient_checkpointing_enable()

args = TrainingArguments(
    output_dir="t5-jfleg",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2,
    eval_strategy="no",
    num_train_epochs=3,
    logging_steps=100,
    save_strategy="steps",
    save_steps=300,
    save_total_limit=2,
    fp16=True,
)
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=ds_tok,
)
trainer.train()
trainer.save_model("t5-jfleg-final")
tokenizer.save_pretrained("t5-jfleg-final")
```

[1134/1134 3:51:26, Epoch 3/3]

| Step | Training Loss |
|------|---------------|
| 100  | 1.803200      |
| 200  | 0.296500      |
| 300  | 0.213700      |
| 400  | 0.156300      |
| 500  | 0.148600      |
| 600  | 0.145600      |
| 700  | 0.150900      |
| 800  | 0.137400      |
| 900  | 0.135000      |
| 1000 | 0.131300      |
| 1100 | 0.132700      |

```
('t5-jfleg-final/tokenizer_config.json',  
 't5-jfleg-final/special_tokens_map.json',  
 't5-jfleg-final/spiece.model',  
 't5-jfleg-final/added_tokens.json')
```

### - Đánh giá và kết quả cuối

Sau huấn luyện, em đánh giá chất lượng bằng cách sinh văn bản với mô hình và so sánh với nhãn đúng. Cụ thể, mã Python định nghĩa hàm `correct_grammar(text)`, trong đó nếu câu không có tiền tố “grammar:” sẽ thêm vào, rồi dùng `model.generate` với beam search (`num_beams=2`) để sinh câu sửa lỗi. Chạy hàm này cho mỗi câu trong tập test (JFLEG test), thu được cột “prediction” chứa các câu được sửa (có thể không hoàn hảo). Cuối cùng, em sử dụng thư viện các thư viện để tính điểm BLEU, Corpus BLEU, Exact Match Accuracy, Avg Levenshtein char distance, Correction F1-score giữa tập văn bản sinh và tập tham chiếu (gồm bốn bản sửa chính xác của mỗi câu).

### Tổng hợp các kết quả lần lượt là:

Corpus BLEU Score: 75.25

Đây là một điểm số BLEU rất tốt (thang điểm 0-100)

Cho thấy văn cảnh và ngữ nghĩa được bảo toàn tốt sau khi sửa

75.25 là điểm số cao trong lĩnh vực GEC, thường các mô hình đạt 60-70

Thể hiện khả năng giữ được cấu trúc câu gốc khi sửa lỗi

Exact Match Accuracy: 0.2233 (22.33%)

Tỉ lệ khá thấp nhưng hợp lý vì:

Một câu có thể có nhiều cách sửa đúng khác nhau

Mô hình có thể sửa đúng về mặt ngữ pháp nhưng không hoàn toàn giống ground truth

Đây là hiện tượng phổ biến trong GEC, thường  $< 30\%$

Average Levenshtein Distance: 14.20

Khoảng cách Levenshtein đo số thao tác sửa đổi ký tự

14.20 là hơi cao, cho thấy mô hình:

Thường sửa đổi nhiều từ trong câu

Có thể đang over-correct

Nên xem xét thêm constraints để giảm số lượng sửa đổi

Correction F1-score: 0.8121 (81.21%)

F1-score rất tốt cho thấy:

Balance tốt giữa precision và recall

Phát hiện chính xác các lỗi cần sửa

Ít bỏ sót lỗi và ít sửa nhầm chỗ đúng

### Nhận xét tổng thể:

Ưu điểm:

BLEU score và F1-score cao cho thấy chất lượng sửa lỗi tốt

Bảo toàn được ngữ nghĩa và văn cảnh của câu gốc

Phát hiện lỗi chính xác

Nhược điểm:

Exact Match thấp - cần cải thiện khả năng match với ground truth

Levenshtein distance cao - cần giảm số lượng sửa đổi

Có thể đang sửa quá nhiều (over-correction)

### D. Kết quả và phân tích

| Mô hình             | Correction F1-score | Avg Levenshtein char distance | Exact Match Accuracy | Corpus BLEU   |
|---------------------|---------------------|-------------------------------|----------------------|---------------|
| RandomForest        | <b>0.8624</b>       | <b>10.5561</b>                | 0.1270               | 0.6644        |
| T5-small(fine-tune) | 0.8121              | 14.20                         | <b>0.2233</b>        | <b>0.7525</b> |

#### 1. F1-score & Levenshtein Distance

RandomForest đạt F1 cao nhất (0.8624) và khoảng cách Levenshtein trung bình thấp nhất (10.56): cho thấy mô hình cây quyết định rất “nhạy” trong việc phát hiện và sửa lỗi (ưu tiên precision + recall).

T5-small fine-tune cũng cho F1 khá tốt (0.8121) nhưng Levenshtein cao hơn (14.2): tức nếu sửa, nó sửa nhiều ký tự hơn (hoặc tạo ra thay đổi dài hơn), có thể sửa “quá tay” một số chỗ.

#### 2. Exact Match Accuracy

T5-small fine-tune: 22.33% câu sinh ra hoàn toàn trùng khớp với bản sửa chuẩn — cao nhất trong ba.



RandomForest: chỉ ~12.7% câu chính xác hoàn toàn, tức dù rất biết “chỗ nào sai” (F1 cao), nhưng phần lớn nó chỉ sửa một phần, không với hết các vị trí sai.

### 3. Corpus-BLEU

T5-small fine-tune dẫn đầu về BLEU (0.7525), tức đầu ra của nó gần gấp ~3/4 n-gram so với bản tham chiếu.

RandomForest đạt BLEU = 0.6644, vẫn tương đối cao với một mô hình không phải seq2seq, nhưng thua T5-small.

### E. Demo

Với việc train 2 mô hình em đã thu được file trọng số của mô hình. Em sẽ trên khai demo theo mô hình t5-small và test 1 vài kết quả xem mô hình đã chạy ổn chưa, đã dự đoán được lỗi sai tiếng anh chưa:

input\_text

Although he enjoy playing football with his friends, he don't like watching it on TV.

Clear

Submit

output

Although he enjoys playing football with his friends, he doesn't like watching it on TV.

Flag

Mô hình đã sửa đúng 2 từ sai là enjoy và don't ( phải chia số ít)

input\_text

The girl which won the contest were very happy and thank everyone for their support.

Clear

Submit

output

The girl who won the contest was very happy and thank everyone for their support.

Flag

Mô hình đã sửa đúng 1 là which -> who 2 là chia lại were nhưng vẫn chưa phát hiện được thank -> thanked

input\_text

She promised helping me with my homework after dinner.

Clear

Example

output

She promised to help me with my homework after dinner.

Flag

Promised đi với to - V và mô hình trên đã nhận diện đúng.

### Nhận xét tổng quan:

- Hiểu tốt ngữ pháp tiếng Anh
- Xử lý được nhiều loại lỗi: mạo từ, thì, chia động từ, giới từ, V-ing/to V,...
- Có khả năng tổng quát tốt, không sửa bậy

### F. Kết luận và định hướng tương lai

#### Nhận xét tổng quan

RandomForest

Ưu điểm: Rất tốt trong việc nhận biết vị trí có lỗi (F1 cao), sửa “có chọn lọc” để giảm số kí tự chỉnh (Levenshtein thấp).

Nhược điểm: Khả năng sinh câu hoàn chỉnh kém (Exact Match thấp, BLEU thua T5). Mô hình kiểu rule-based/feature-based đôi khi chỉ sửa lẻ tẻ từ đơn, không bắt kịp ngữ cảnh rộng.

## T5- small fine- tune

Ưu điểm: Kết hợp tốt giữa nhận biết và sinh sửa: F1 cao, Exact Match tốt nhất và BLEU cao nhất. Mô hình pre- trained giúp nắm bắt ngữ cảnh rộng, lưu loát hơn.

Nhược điểm: Levenshtein lớn nghĩa là đôi khi sửa “đồ sộ” hơn cần thiết, có thể làm thay đổi cấu trúc câu nhiều.

### **Nguồn gốc điểm yếu**

#### **RandomForest**

##### **Điểm yếu**

Thiếu ngữ cảnh dài hạn: RandomForest dựa trên các đặc trưng cục bộ (n-gram, POS tag, rule...), nên rất khó hiểu toàn bộ cấu trúc câu để sửa những lỗi phức tạp liên quan tới trật tự từ hay mệnh đề phụ.

Chỉ sửa được lỗi đơn lẻ: Mô hình dễ “nhận diện” vị trí có lỗi (F1 cao) nhưng mỗi lần chỉ sửa vài token chứ không tái cấu trúc cả câu. Do đó BLEU và Exact Match thấp hơn T5.

Overfitting feature-based: Với dữ liệu nhỏ và quá nhiều đặc trưng thủ công, mô hình có thể ghi nhớ “rule” cụ thể mà không khái quát được.

##### **Cách cải thiện**

Thêm ngữ cảnh: bổ sung các đặc trưng trình tự dài hơn (dependency features, embeddings từ pre- trained) trước khi cho vào RandomForest.

Ensemble: kết hợp với mô hình seq2seq (ví dụ T5) – dùng RandomForest làm bộ lọc chỗ cần sửa, rồi để T5-chỉnh tiếp.

Giảm dimensionality: dùng PCA hoặc embedding giảm kích thước feature để tránh overfit, và thêm cross- validation chặt hơn.

#### **T5- small**

##### **Điểm yếu**

Sửa “quá tay” (High Levenshtein distance): mô hình chỉnh nhiều token hơn cần thiết, có thể vì fine-tune với threshold low khiến nó “dập” lại cả cụm.

Plateau sớm: dù pre- trained nhưng fine-tune với tập dữ liệu nhỏ (~1–2K câu) nên đến epoch 3–4 đã bão hòa.

Thiếu đa dạng trong decoding: mặc định beam search size nhỏ (4) và không có penalty length, dẫn đến kết quả tương tự nhau, ít cải thiện BLEU sau này.

##### **Cách cải thiện**

Dữ liệu mở rộng: kết hợp thêm corpus GEC khác (CoNLL, Lang-8...) hoặc augment (back- translation, paraphrasing) để tăng số mẫu sửa.

Tinh chỉnh hyper- parameters:

Giảm learning rate (vd.  $1e-5$ ), tăng epochs nhẹ nhàng với early- stopping.

Thử label smoothing (~0.1) để mô hình mềm dẻo hơn, không “dập” quá mạnh.

Điều chỉnh decode:

Tăng beam size (8–10), hoặc dùng diverse beam search.

Thử length penalty ( $>1$ ) để tránh cắt cụm sửa quá dài.

Layer unfreezing dần: ban đầu chỉ fine-tune lớp cuối, sau vài epoch mở thêm encoder layers để tránh catastrophic forgetting.

## Tổng kết

Trong bài toán sửa lỗi ngữ pháp tiếng Anh, ba hướng tiếp cận—RandomForest, T5-small fine-tune và đều cho thấy những ưu, nhược điểm tương ứng. Mặc dù RandomForest thể hiện khả năng phát hiện lỗi tốt nhất (F1 cao), nhưng thiếu ngữ cảnh sâu để sinh câu hoàn chỉnh. T5-small fine-tune khai thác hiệu quả kiến thức ngôn ngữ từ pre-trained, dẫn đến kết quả BLEU và Exact Match ấn tượng, dù vẫn cần tinh chỉnh hyper-parameters và cách sinh (decoding) để giảm sửa “quá tay.” Cuối cùng, mô hình xây từ đầu trên dữ liệu hạn chế chưa đủ sức học, minh chứng cho tầm quan trọng của việc tận dụng mô hình pretrained và dữ liệu lớn.

Hướng phát triển tiếp theo bao gồm:

Mở rộng dữ liệu: kết hợp thêm các corpus GEC khác và các kỹ thuật data augmentation.

Fine-tune mô hình pretrained đầy đủ (T5, BART), thay vì train from scratch.

Tinh chỉnh quá trình huấn luyện (learning rate schedule, early-stopping, regularization) và decode strategy (beam size, length penalty) để khai thác tối đa tiềm năng mô hình.

Kết hợp ensemble giữa các mô hình để tận dụng điểm mạnh riêng—ví dụ dùng RandomForest phát hiện lỗi, rồi để T5 điều chỉnh chi tiết.

Với những bước đi này, dự án sửa lỗi ngữ pháp có thể đạt được độ chính xác, độ lưu loát và khả năng áp dụng thực tiễn cao hơn.

## Lời kết

Qua hành trình nghiên cứu và thử nghiệm ba phương pháp—from mô hình đặc trưng cục bộ RandomForest, đến fine-tune T5-small trên kiến thức ngôn ngữ tiền huấn luyện, chúng ta đã rút ra được bài học quan trọng:

**chất lượng và quy mô dữ liệu, cùng tiềm năng từ các mô hình pre-trained**, quyết định phần lớn hiệu năng sửa lỗi ngữ pháp.

RandomForest nhanh gọn trong phát hiện lỗi, T5-small cho kết quả lưu loát và chính xác cao hơn, trong khi nỗ lực train from scratch chưa thể vượt qua giới hạn dữ liệu. Để tiến xa hơn, cần tiếp tục mở rộng corpus sửa lỗi, tận dụng sức mạnh của mô hình pretrained đầy đủ (T5, BART...), đồng thời tinh chỉnh chiến lược huấn luyện và giải mã. Với lộ trình đó, tin rằng hệ thống sửa lỗi ngữ pháp sẽ ngày càng hoàn thiện, đáp ứng tốt nhu

cầu ứng dụng thực tế và hỗ trợ người dùng nâng cao chất lượng văn bản tiếng Anh.

Dataset: [jhu-clsp/jfleg](#) • [Datasets at Hugging Face](#)

Nguồn tham khảo:

[1] : [T5](#)

[2] : [Transformer \(deep learning architecture\) - Wikipedia](#)

[3] : [AliHaiderAhmad001/T5-Relative-Position:](#)

[Implementation for the Relative Position model introduced by T5 \(Raffel et al. 2020\) Alongside a Comprehensive Demonstration.](#)

[Attention \(machine learning\) - Wikipedia](#)

[BLEU - Wikipedia](#)

[ROUGE \(metric\) - Wikipedia](#)